

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Wydział Informatyki
Filia w Gdańsku

Otrombke Natalia
Nr albumu s26828
Nazwa specjalizacji: Cyberbezpieczeństwo

Kasprowicz Oliwier
Nr albumu s30290
Nazwa specjalizacji: Cyberbezpieczeństwo

Gutowski Michał
Nr albumu s30572
Nazwa specjalizacji: Cyberbezpieczeństwo

Parkitny Miłosz
Nr albumu s30287
Nazwa specjalizacji: Cyberbezpieczeństwo

Wronisz Wojciech
Nr albumu s26999
Nazwa specjalizacji: Cyberbezpieczeństwo

RacePortal — platforma wydarzeń motorsportowych

Rodzaj pracy
inżynierska

Imię i nazwisko promotora
dr hab. Marek Bednarczyk

Gdańsk, miesiąc, rok obrony

Streszczenie: Praca przedstawia projekt i implementację systemu RacePortal — platformy wspierającej katalogowanie wydarzeń motorsportowych oraz proces zapisów uczestników. System obejmuje aplikację webową, backend REST oparty o Spring Boot i MySQL oraz aplikację mobilną Expo pełniącą rolę publicznej wizytówki katalogu. W pracy opisano problem dziedzinowy, wymagania, architekturę, realizację kluczowych modułów (wydarzenia, garaż, zgłoszenia, role), mechanizmy bezpieczeństwa (JWT, RBAC) oraz weryfikację testami automatycznymi. Wynikiem jest działające, konteneryzowane środowisko demonstracyjne zgodne z zakresem MVP projektu.
Słowa kluczowe: motorsport, katalog wydarzeń, system zapisów, zarządzanie wydarzeniami, aplikacja mobilna

POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

Karta projektu

Temat projektu: RacePortal — platforma wydarzeń motorsportowych **Temat projektu po angielsku:** RacePortal — a platform for motorsports events	**Akronim:** RP **Data ustalenia tematu** 2025-10-01
Promotor: dr hab. Marek Bednarczyk	**Konsultanci:** 1. — brak —
Cele projektu: Celem projektu jest zaprojektowanie i zaimplementowanie systemu RacePortal — centralnego katalogu wydarzeń motorsportowych w Polsce, umożliwiającego przeglądanie wydarzeń (lista, mapa, kalendarz), zarządzanie garażem pojazdów, zgłoszenia udziału oraz obsługę ról organizatora i administratora, wraz z aplikacją mobilną typu wizytówka oraz konteneryzowanym środowiskiem uruchomieniowym.	
Rezultaty projektu: Aplikacja webowa (React/Vite), API REST (Spring Boot) z bazą MySQL, aplikacja mobilna Expo (katalog wydarzeń z CTA do weba), wspólne typy API (`packages/api-types`), dokumentacja projektowa oraz zestaw testów automatycznych (JUnit, Vitest, Playwright). Katalog wydarzeń z filtrami oraz widokami lista/mapa/kalendarz; szczegóły wydarzenia i wyznaczanie trasy (OSRM); konta użytkowników (JWT, RBAC); garaż pojazdów i dopasowanie kategorii do wydarzenia (`carMatch`); system zgłoszeń ze statusami i powiadomieniami e-mail; panele organizatora i administratora; archiwum; mobilna wizytówka z przekierowaniem zapisu na web.	
Miary sukcesu: Działający stos Docker Compose (web, API, MySQL, backup) z SMTP Mailbux; realizacja funkcji MVP katalogu, zapisów i ról; pozytywne wyniki testów automatycznych API/web/mobile unit; możliwość demonstracji pełnego scenariusza: gość → kierowca → organizator → administrator oraz mobilka → CTA web.	
Ograniczenia: Brak natywnej bramki płatności (PSP); aplikacja mobilna bez pełnej równoważności funkcji z webem (świadoma wizytówka — bez logowania, garażu i uploadu); brak publikacji w sklepach App Store/Google Play; brak formalnego testu obciążenia 10k rekordów / 50 RPS (są natomiast krótkie pomiary k6 na VM); produkcyjny TLS poza domkniętym zakresem lokalnego MVP (w prod: Cloudflare Tunnel).	

Wykonawcy	**Numer albumu**	**Specjalizacja**	**Tryb studiów**
Otrombke Natalia	s26828	Cyberbezpieczeństwo	Stacjonarny
Kasprowicz Oliwier	s30290	Cyberbezpieczeństwo	Stacjonarny
Gutowski Michał	s30572	Cyberbezpieczeństwo	Stacjonarny
Parkitny Miłosz	s30287	Cyberbezpieczeństwo	Stacjonarny
Wronisz Wojciech	s26999	Cyberbezpieczeństwo	Stacjonarny

Data ukończenia projektu:	**Recenzent:**
7 listopada 2026	mgr Alex Polin

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Otrombke Natalia**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26828**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Kasprowicz Oliwier**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30290**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Gutowski Michał**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30572**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Parkitny Miłosz**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s30287**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

. . . . . .

(data, miejscowość)

OŚWIADCZENIE STUDENTA
DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: **Wronisz Wojciech**

student/ka kierunku: **Cyberbezpieczeństwo**

numer albumu studenta: **s26999**

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK", stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

. . . . . .

(czytelny podpis studenta/studentki)

Spis treści

1 Informacje wstępne **23**

1.1 Cel i zakres pracy 23

1.2 Wizja rozwiązania 24

1.3 Struktura pracy 24

2 Słownik pojęć **27**

3 Opis problemu **29**

3.1 Charakterystyka rynku wydarzeń motorsportowych 29

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników 29

3.3 Problemy informacyjne po stronie kierowców i organizatorów 30

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal 30

3.5 Aspekty techniczne i organizacyjne problemu 31

4 Kontekst biznesowy **32**

4.1 Segmentacja użytkowników 32

4.2 Interesariusze systemu 34

4.3 Propozycja wartości systemu 35

4.4 Kanały komunikacji i dystrybucji 37

4.5 Kluczowe zasoby i działania 37

4.6 Model działania platformy 38

4.7 Realistyczność wdrożenia i dalsze perspektywy 38

4.8 Kontekst prawny i etyczny 39

5 Analiza SWOT **41**

5.1 Mocne strony (Strengths) 41

5.2 Słabe strony (Weaknesses) 42

5.3 Szanse (Opportunities) 42

5.4 Zagrożenia (Threats) 43

5.5 Podsumowanie analizy SWOT 43

6 Planowanie projektu **45**
6.1 Cele projektu . . . 45
6.2 Zakres projektu . . . 46
6.3 Metodyka pracy . . . 47
6.4 Organizacja zespołu . . . 48
6.5 Harmonogram pracy . . . 48
6.6 Infrastruktura i środowisko pracy . . . 49
6.7 Analiza ryzyka projektowego . . . 50
6.7.1 Analiza zagrożeń . . . 50

7 Analiza wymagań **52**
7.1 Źródła i sposób pozyskania wymagań . . . 52
7.2 Wymagania ogólne i dziedzinowe . . . 53
7.3 Wymagania funkcjonalne . . . 54
7.4 Wymagania niefunkcjonalne . . . 56
7.5 Wymagania bezpieczeństwa . . . 57
7.6 Wymagania interfejsu użytkownika . . . 58
7.7 Diagramy UML . . . 59
7.7.1 Diagram przypadków użycia . . . 59
7.7.2 Diagram sekwencji: rejestracja i logowanie . . . 59
7.7.3 Diagram sekwencji: zapis na wydarzenie . . . 59
7.7.4 Diagram sekwencji: zarządzanie garażem . . . 59

8 Projekt systemu **60**
8.1 Założenia architektoniczne . . . 60
8.2 Architektura logiczna systemu . . . 61
8.3 Architektura fizyczna i wdrożeniowa . . . 61
8.4 Projekt bazy danych . . . 62
8.4.1 Diagram klas . . . 62
8.5 Projekt interfejsu REST API . . . 63
8.6 Projekt warstwy webowej . . . 64
8.6.1 Prototyp UI w Figma Make . . . 66
8.7 Projekt aplikacji mobilnej . . . 67
8.8 Projekt mechanizmów bezpieczeństwa . . . 67

9 Realizacja **69**
9.1 Organizacja implementacji . . . 69
9.2 Podział systemu na komponenty . . . 69
9.2.1 Backend . . . 70

9.2.2 Frontend webowy 70
9.2.3 Aplikacja mobilna 71
9.3 Integracja komponentów 71
9.4 Konteneryzacja i uruchamianie środowiska 71

10 Szczegóły implementacji backendu **73**
10.1 Struktura projektu backendowego 73
10.2 Model domenowy i encje 74
10.3 Rejestracja użytkowników i logowanie 74
10.4 Zarządzanie profilami użytkowników 75
10.5 Obsługa wydarzeń 76
10.6 Obsługa zgłoszeń 77
10.7 Obsługa garażu 78
10.8 Panel administratora i organizatora 78
10.9 Integracja z bazą danych 79
10.10 Obsługa wyjątków i walidacja 80

11 Szczegóły implementacji frontendu webowego **82**
11.1 Struktura aplikacji webowej 82
11.2 Strona główna i katalog wydarzeń 83
11.3 Widok szczegółów wydarzenia 83
11.4 Logowanie i rejestracja 83
11.5 Dashboard użytkownika 84
11.6 Widok garażu 84
11.7 Panel organizatora 85
11.8 Panel administratora 85
11.9 Mapa i elementy lokalizacyjne 85
11.10 Responsywność i doświadczenie użytkownika 85

12 Szczegóły implementacji aplikacji mobilnej **86**
12.1 Założenia aplikacji mobilnej 86
12.2 Architektura rozwiązania mobilnego 87
12.3 Lista wydarzeń i widok szczegółów 87
12.4 Nawigacja i interfejs użytkownika 88
12.5 Integracja z API 89
12.6 Przekierowanie do wersji webowej 90
12.7 Ograniczenia wersji mobilnej 90

13 Bezpieczeństwo systemu **91**
13.1 Założenia bezpieczeństwa 91
13.2 Klasyfikacja danych 91
13.3 Uwierzytelnianie użytkowników 91
13.4 Autoryzacja i kontrola dostępu RBAC 91
13.5 Wykorzystanie tokenów JWT 91
13.6 Walidacja danych wejściowych 91
13.7 Ochrona zasobów użytkowników 92
13.8 Bezpieczeństwo komunikacji 92
13.9 Analiza zagrożeń 92
13.10 Analiza ryzyka 92
13.11 Ograniczenia przyjętych zabezpieczeń 92

14 Testowanie i weryfikacja systemu **93**
14.1 Strategia testowania 93
14.2 Środowisko testowe 93
14.3 Testy jednostkowe 93
14.4 Testy integracyjne backendu 93
14.5 Testy End-to-End aplikacji webowej 93
14.6 Testy aplikacji mobilnej 93
14.7 Testy mechanizmów bezpieczeństwa 93
14.8 Automatyzacja uruchamiania testów 94
14.9 Analiza wyników testów 94
14.10 Ograniczenia procesu testowania 94

15 Prezentacja systemu **95**
15.1 Prezentacja głównych funkcjonalności 95
15.2 Scenariusz użycia z perspektywy kierowcy 95
15.3 Scenariusz użycia z perspektywy organizatora 95
15.4 Scenariusz użycia z perspektywy administratora 95
15.5 Podsumowanie działania systemu 95

16 Nakład pracy **96**
16.1 Liczba godzin przeznaczona na realizację projektu 96
16.2 Podział zadań w zespole 96
16.3 Narzędzia wspierające zarządzanie pracą 96
16.4 Problemy organizacyjne i ich rozwiązanie 96

17 Wkład własny **97**
17.1 Natalia Otrombke 97
17.2 Oliwier Kasprowicz 97
17.3 Michał Gutowski 97
17.4 Miłosz Parkitny 98
17.5 Wojciech Wronisz 98

18 Podsumowanie **99**
18.1 Największe wyzwania i sposób ich rozwiązania 99
18.2 Zmiany i adaptacje w trakcie realizacji 100
18.3 Ograniczenia końcowej wersji systemu 100
18.3.1 Ograniczenia wydajnościowe (RPS) 101
18.4 Możliwości dalszego rozwoju 102
18.5 Wnioski końcowe 103

Bibliografia **104**

Spis rysunków **105**

Spis tabel **106**

Załączniki **108**

Rozdział 1

Informacje wstępne

1.1 Cel i zakres pracy

Celem pracy jest zaprojektowanie oraz implementacja systemu **RacePortal** — platformy internetowej (z uzupełnieniem mobilnym) służącej jako centralny katalog wydarzeń motorsportowych w Polsce oraz narzędzie do obsługi zapisów kierowców. Projekt stanowi odpowiedź na rosnące zapotrzebowanie społeczności motorsportowej na ujednolicone narzędzie eliminujące rozproszenie informacji na grupach społecznościowych i niespójnych stronach organizatorów.

Zakres projektu obejmuje:

- **Aplikację webową** — kompleksowy interfejs użytkownika zrealizowany w architekturze SPA (React/Vite), obsługujący moduły katalogu wydarzeń, kont użytkowników, wirtualnego garażu pojazdów, mechanizmu dopasowania kategorii (`carMatch`), zgłoszeń na imprezy oraz dedykowane panele zarządzania dla organizatorów i administratorów.
- **API REST oraz warstwę danych** — backend wykonany w oparciu o framework Spring Boot oraz relacyjną bazę danych MySQL (z wydzielonymi strukturami encji i wspólnymi typami API w pakiecie `packages/api-types`), zapewniający bezpieczne uwierzytelnianie (JWT) oraz kontrolę dostępu opartą o role (RBAC: `USER`, `ORGANIZER`, `ADMIN`).
- **System powiadomień i map** — zautomatyzowane powiadomienia e-mail (realizowane m.in. przez serwer SMTP Mailbux) dotyczące potwierdzeń rejestracji, statusów zgłoszeń oraz weryfikacji kont, a także integrację map wydarzeń z usługą wyznaczania tras (domyślnie OSRM z możliwością rozszerzenia o Google Maps API).
- **Aplikację mobilną (Expo)** — lekką, wieloplatformową aplikację mobilną opartą na ekosystemie Expo, pełniącą rolę mobilnej wizytówki katalogu z czytelnym mechanizmem konwersji (Call to Action — CTA) przekierowującym użytkownika do pełnego serwisu webowego.

- **Infrastrukturę i testowanie** — pełną konteneryzację całego środowiska uruchomieniowego przy użyciu Docker Compose (web, API, baza danych, backupy) z zabezpieczeniem produkcyjnego ruchu przez Cloudflare Tunnel oraz przygotowanie podstawowego zestawu testów automatycznych (jednostkowych i integracyjnych).

Poza zakresem MVP pozostają m.in. natywne płatności online, push notifications oraz publikacja aplikacji mobilnej w sklepach. Upload plików (obrazy JPG/PNG/WebP oraz PDF) jest dostępny po stronie **weba** i API (`/api/uploads/...`); aplikacja mobilna Expo nie udostępnia własnego formularza wgrywania — jedynie wyświetla obrazy już opublikowane w katalogu.

1.2 Wizja rozwiązania

RacePortal ma być **jednym miejscem** odkrywania i obsługi startów motorsportowych w Polsce: kierowca znajduje wydarzenie, sprawdza wymagania względem własnego garażu i składa zgłoszenie; organizator publikuje imprezę i zarządza listą uczestników; administrator moderuje treści i role.

Wizja produktowa zakłada:

- publiczny katalog z trzema widokami (lista, mapa, kalendarz) na wspólnych filtrach;
- ujednolicony proces zapisu ze statusami i powiadomieniami e-mail;
- wyraźny podział ról `USER` / `ORGANIZER` / `ADMIN` (gość przegląda katalog bez konieczności zakładania konta);
- mobilną ścieżkę discovery (Expo) z konwersją zapisu na wersję webową;
- uruchamialne środowisko demonstracyjne (Docker Compose) oraz wdrożenie produkcyjne realizowane za pośrednictwem Cloudflare Tunnel.

Szczegóły warstw technicznych (web React [1]/Vite, API Spring Boot [2], MySQL [3], mobile Expo [4], `packages/api-types`) opisano w rozdziale 8 oraz w rozdziałach implementacyjnych. Kod źródłowy utrzymywany jest w repozytorium projektu [5].

1.3 Struktura pracy

Niniejsza praca inżynierska została podzielona na logiczne i spójne rozdziały, które w sposób wyczerpujący przeprowadzają czytelnika przez proces koncepcyjny, analityczny, projektowy oraz implementacyjny systemu RacePortal:

- **Rozdział 1 (Informacje wstępne)** — przedstawia cel i zakres pracy, wizję rozwiązania opartą na architekturze MVP, a także strukturę całego dokumentu.

- **Rozdział 2 (Słownik pojęć)** — definiuje kluczowe terminy dziedzinowe i technologiczne wykorzystane w pracy, takie jak wydarzenie, katalog, garaż, zgłoszenie, mechanizm dopasowania pojazdu (`carMatch`), a także pojęcia związane z bezpieczeństwem i architekturą (JWT, RBAC, OSRM, MVP).

- **Rozdział 3 (Opis problemu)** — charakteryzuje specyfikę rynku amatorskiego i półprofesjonalnego motorsportu w Polsce, wskazuje na problemy organizacyjne i informacyjne po stronie kierowców oraz organizatorów, a także uzasadnia potrzebę stworzenia zintegrowanej platformy.

- **Rozdział 4 (Kontekst biznesowy)** — omawia segmentację użytkowników, mapę interesariuszy, propozycję wartości systemu, kanały komunikacji, model działania platformy oraz uwarunkowania prawne i etyczne.

- **Rozdział 5 (Analiza SWOT)** — analizuje mocne i słabe strony projektu oraz identyfikuje rynkowe szanse i zagrożenia związane z wdrożeniem systemu.

- **Rozdział 6 (Planowanie projektu)** — przedstawia cele, zakres, metodykę pracy, organizację zespołu, harmonogram, zastosowaną infrastrukturę oraz szczegółową analizę ryzyka projektowego.

- **Rozdział 7 (Analiza wymagań)** — zawiera źródła pozyskania wymagań, wymagania ogólne, dziedzinowe, funkcjonalne, niefunkcjonalne, bezpieczeństwa oraz interfejsu użytkownika, a także diagramy UML (w tym przypadki użycia).

- **Rozdział 8 (Projekt systemu)** — omawia założenia architektoniczne, architekturę logiczną i fizyczną, projekt relacyjnej bazy danych (model ERD), projekt interfejsu REST API, warstwy webowej (Figma Make), aplikacji mobilnej oraz mechanizmów bezpieczeństwa.

- **Rozdział 9 (Realizacja)** — przedstawia organizację prac implementacyjnych, podział systemu na komponenty (backend Spring Boot, frontend React/Vite, aplikacja mobilna Expo), integrację oraz konteneryzację środowiska za pomocą Docker Compose.

- **Rozdział 10 (Szczegóły implementacji backendu)** — szczegółowo opisuje strukturę projektu backendowego, model domenowy, mechanizmy uwierzytelniania i rejestracji (JWT, rate limiting), obsługę wydarzeń, zgłoszeń, garażu oraz integrację z bazą danych.

- **Rozdział 11 (Szczegóły implementacji frontendu webowego)** — omawia strukturę aplikacji webowej, kluczowe widoki (katalog, szczegóły, dashboard użytkownika, garaż, panele administratora i organizatora) oraz responsywność i doświadczenie użytkownika.

- **Rozdział 12 (Szczegóły implementacji aplikacji mobilnej)** — przedstawia założenia, architekturę oraz implementację mobilnej wizytówki systemu opartej na ekosystemie Expo.
- **Rozdział 13 (Bezpieczeństwo systemu)** — prezentuje założenia ochrony danych, klasyfikację informacji, uwierzytelnianie, kontrolę dostępu (RBAC), walidację wejścia oraz analizę zagrożeń i ryzyka.
- **Rozdział 14 (Testowanie i weryfikacja systemu)** — opisuje strategię testowania, środowisko testowe, testy jednostkowe, integracyjne, E2E (Playwright) oraz weryfikację mechanizmów bezpieczeństwa.
- **Rozdział 15 (Prezentacja systemu)** — ilustruje scenariusze użycia platformy z perspektywy kierowcy, organizatora oraz administratora.
- **Rozdział 16 (Nakład pracy)** — podsumowuje liczbę przepracowanych godzin, podział zadań w zespole oraz narzędzia wspierające zarządzanie pracą.
- **Rozdział 17 (Wkład własny)** — szczegółowo opisuje indywidualny wkład poszczególnych członków zespołu w realizację projektu.
- **Rozdział 18 (Podsumowanie)** — przedstawia największe wyzwania, adaptacje w trakcie prac, końcowe ograniczenia systemu (w tym wydajnościowe RPS) oraz perspektywy dalszego rozwoju platformy.

Rozdział 2

Słownik pojęć

Wydarzenie (event) Zorganizowana impreza motorsportowa (np. track day, drift, rally), opisana datą, lokalizacją, kategorią i wymaganiami udziału.

Katalog Publiczny wykaz wydarzeń z widokami lista, mapa i kalendarz oraz filtrami.

Garaż Zbiór pojazdów przypisanych do konta kierowcy, wykorzystywanych przy zapisie na wydarzenie.

Zgłoszenie (registration) Wniosek o udział w wydarzeniu. Statusy dyplomowe: `PENDING` (oczekuje), `ACCEPTED` (wstępnie przyjęte), `CONFIRMED` (potwierdzone), `CANCELED` (anulowane).

Model „zapłać później” Ścieżka płatnego wydarzenia bez bramki online: po `ACCEPTED` kierowca dokłada dowód przelewu, a organizator potwierdza miejsce (`CONFIRMED`).

carMatch Mechanizm dopasowania kategorii / klasy pojazdu do wymagań wydarzenia (m.in. aliasy nazw).

Workspace organizatora Panel zarządzania pojedynczym wydarzeniem (zgłoszenia, klasy, wyniki, komunikacja, wskaźniki).

Lista startowa Publiczny wykaz potwierdzonych uczestników wydarzenia (po stronie weba).

Organizator Użytkownik z rolą `ORGANIZER`, uprawniony do tworzenia i zarządzania własnymi wydarzeniami oraz zgłoszeniami.

Administrator Użytkownik z rolą `ADMIN`, moderujący wydarzenia, wnioski o rolę organizatora oraz role kont.

Backoffice Osobny host administracyjny (`bo.raceportal.pl`) przeznaczony wyłącznie dla roli `ADMIN`.

Wizytówka mobilna Aplikacja Expo ograniczona do przeglądania katalogu; zapis i konto realizowane są w wersji webowej.

CTA Call to Action — wezwanie do działania; w mobilce przycisk otwierający zapis na stronie webowej.

PWA Progressive Web App — aplikacja webowa instalowalna na urządzeniu (manifest, service worker), uzupełniająca ścieżkę mobilną obok Expo.

API REST Interfejs HTTP backendu RacePortal (JSON) współdzielony przez web i mobile.

JWT JSON Web Token — token dostępu API; w warstwie webowej w ciasteczkach httpOnly (`rp_at` / `rp_rt`), nie w `localStorage`.

RBAC Role-Based Access Control — kontrola dostępu oparta o role (`USER` / `ORGANIZER` / `ADMIN`).

OAuth Google Opcjonalne logowanie kontem Google (obok logowania hasłem).

OSRM Open Source Routing Machine — publiczny serwis wyznaczania trasy dojazdu (domyślny provider).

Docker Compose Narzędzie uruchamiania zestawu kontenerów (web, API, MySQL, backup) jako wspólnego środowiska lokalnego i produkcyjnego.

Cloudflare Tunnel Tunel zapewniający publiczny dostęp HTTPS do usług na maszynie wirtualnej (bez otwierania portów na firewallu VM).

Mailbux Zewnętrzny SMTP używany do wysyłki maili transakcyjnych RacePortal.

PSP Payment Service Provider — natywna bramka płatności online; świadomie poza zakresem MVP.

MVP Minimum Viable Product — minimalny zakres funkcji przyjęty do odbioru projektu.

RPS Requests Per Second — metryka obciążenia (liczba żądań na sekundę); w projekcie mierzona m.in. narzędziem k6.

Expo Framework do budowy aplikacji React Native; wykorzystany w kliencie mobilnym RacePortal.

Rozdział 3

Opis problemu

3.1 Charakterystyka rynku wydarzeń motorsportowych

Amatorski i półprofesjonalny motorsport w Polsce przeżywa w ostatnich latach dynamiczny rozwój. Obejmuje on liczne, zróżnicowane formy aktywności, takie jak wydarzenia typu *track day*, zawody w drifcie, rajdy amatorskie (KJS, Super OES), wyścigi górskie czy zmagania *time attack*. Infrastruktura, od dużych obiektów pokroju Toru Poznań czy Silesia Ring, po mniejsze tory szkoleniowe i lokalne obiekty, pozwala na organizację imprez o różnym stopniu zaawansowania.

Mimo rosnącej popularności tej formy spędzania czasu, rynek ten charakteryzuje się dużym rozproszeniem organizacyjnym. Informacje o terminach, regulaminach oraz procedurach rejestracji są często rozrzucone pomiędzy indywidualnymi stronami internetowymi organizatorów, fanpage'ami w mediach społecznościowych (głównie na platformie Facebook) oraz tematycznymi forami dyskusyjnymi. Brakuje spójnego, scentralizowanego ekosystemu, który agregowałby tego typu aktywności w sposób ustandaryzowany, ułatwiając tym samym obieg informacji.

3.2 Problemy związane z organizacją wydarzeń i zapisami uczestników

Z perspektywy organizatorów, zarządzanie mniejszymi i średnimi wydarzeniami motorsportowymi wiąże się z szeregiem wyzwań logistycznych. Większość podmiotów wykorzystuje do zbierania zgłoszeń podstawowe i niepołączone ze sobą narzędzia, takie jak formularze Google, arkusze kalkulacyjne czy wiadomości e-mail.

Ręczne przetwarzanie zgłoszeń jest nie tylko czasochłonne, ale i wysoce podatne na błędy ludzkie (np. zgubienie zgłoszenia, błędne przypisanie do klasy startowej). Poważnym problemem jest również weryfikacja techniczna uczestników — organizator musi ręcznie oceniać, czy dany pojazd spełnia wymagania techniczne dla danej kategorii wydarzenia (np. pojemność sil-

nika, napęd, klatka bezpieczeństwa). Dodatkowo, komunikacja z uczestnikami odnośnie statusu ich zgłoszenia (oczekujące, zatwierdzone, odrzucone) wymaga indywidualnej wysyłki wiadomości, co staje się trudne do skalowania przy rosnącej liczbie uczestników na listach startowych i rezerwowych.

3.3 Problemy informacyjne po stronie kierowców i organizatorów

Rozproszenie danych o imprezach dotkliwie uderza również w samych uczestników. Kierowcy mają ogromną trudność ze znalezieniem aktualnych wydarzeń startowych w jednym miejscu. Aby zaplanować sezon lub chociażby weekendowy wyjazd na tor, muszą oni na bieżąco monitorować wiele niezależnych źródeł.

Często proces poszukiwania informacji sprowadza się do przedzierania się przez nieczytelne posty w mediach społecznościowych, gdzie kluczowe dane, takie jak koszty wpisowego, harmonogram czy limit miejsc, są trudne do odnalezienia. Następnie, uczestnik staje przed problemem oceny, czy jego samochód w ogóle kwalifikuje się do wybranej imprezy. Brak centralnego kalendarza oraz ujednoliconego, przejrzystego interfejsu sprawia, że cały proces — od znalezienia wydarzenia, po udany start — jest niepotrzebnie skomplikowany, a komunikacja na linii organizator–kierowca jest chaotyczna i opóźniona.

3.4 Uzasadnienie potrzeby stworzenia systemu RacePortal

Projekt platformy RacePortal jest bezpośrednią odpowiedzią na wyżej zidentyfikowane problemy. Jego głównym założeniem jest dostarczenie **centralnego katalogu wydarzeń** oraz zautomatyzowanego, **ujednoliconego procesu zapisów**, w którym obie strony (kierowca i organizator) operują na tym samym zbiorze i formacie danych.

Stworzenie dedykowanego rozwiązania pozwoli na:

- zebranie wszystkich typów wydarzeń motorsportowych w łatwej do przeszukiwania bazie z widokiem mapy, listy i kalendarza;
- wprowadzenie mechanizmu „wirtualnego garażu” dla uczestników, z poziomu którego ich pojazdy mogą być automatycznie zestawiane z wymogami wydarzeń (mechanizm *carMatch*);
- uporządkowanie ról w systemie (Gość, Użytkownik/Kierowca, Organizator, Administrator), co usprawni przepływ informacji, walidację zgłoszeń oraz moderację treści;
- zapewnienie użytkownikom dostępu poprzez ścieżkę webową (pełna funkcjonalność) oraz mobilną wizytówkę (ułatwione odkrywanie wydarzeń z poziomu smartfona).

3.5 Aspekty techniczne i organizacyjne problemu

Rozwiązanie zdefiniowanych problemów biznesowych pociąga za sobą konieczność sprostania konkretnym wyzwaniom technicznym. Głównym zadaniem jest zaprojektowanie wydajnego i spójnego modelu relacyjnej bazy danych, który sprawnie powiąże wydarzenia, użytkowników, pojazdy z ich parametrami oraz listę zgłoszeń i ich statusy.

Kluczowe zagadnienia techniczne podjęte w ramach projektu obejmują:

- **Bezpieczeństwo i autoryzację:** implementację bezstanowego uwierzytelniania opartego o tokeny JWT oraz ścisłej, opartej na rolach kontroli dostępu (RBAC) do poszczególnych zasobów API;
- **Walidację logiczną:** realizację mechanizmów sprawdzających wymagania wydarzenia względem pojazdu (moduł *carMatch*);
- **Niezawodną komunikację asynchroniczną:** wdrożenie systemu powiadomień e-mail dostarczającego uczestnikom zmiany statusów, również w środowiskach izolowanych.

Rozdział 4

Kontekst biznesowy

Niniejszy rozdział sytuuje RacePortal w kontekście produktowym i organizacyjnym: identyfikuje segmenty odbiorców, udziałowców oraz propozycję wartości, a następnie opisuje kanały dystrybucji, zasoby i działania kluczowe, model funkcjonowania platformy, realność wdrożenia oraz uwarunkowania prawno-etyczne. Punktem wyjścia są założenia Dokumentu Założeń Wstępnych (DZW), karty projektu, Business Model Canvas (BMC / materiały KMR) oraz scenariusze walidacji wywiadami, konfrontowane z faktycznym zakresem rozwiązania.

Należy zaznaczyć, że segmentacja w materiałach KMR została sformułowana jako *założenie koncepcyjne*, a nie jako wynik reprezentatywnych badań rynkowych. Dokument *Interview Validation* definiuje hipotezy, skrypt wywiadu oraz metryki sukcesu; nie zawiera wyników przeprowadzonych wywiadów, dlatego w dalszej części nie są one przytaczane jako dane empiryczne. Analogicznie, wskazany w materiałach koncepcyjnych przedział wiekowy 16–40 lat traktuje się wyłącznie jako hipotezę dotyczącą grupy docelowej, a nie jako potwierdzoną charakterystykę demograficzną rynku.

4.1 Segmentacja użytkowników

Z perspektywy biznesowej wyróżnia się trzy główne segmenty rynku, których potrzeby determinują wartość RacePortal. Segmentację rynkową należy odróżnić od ról dostępowych w systemie (kontrola dostępu oparta o role): role techniczne opisują uprawnienia w aplikacji, natomiast segmenty opisują grupy odbiorców i ich problemy.

Odbiorcy informacji (kibice i pasjonaci) poszukują aktualnych terminów, lokalizacji oraz podstawowych informacji o wydarzeniach motorsportowych. Dla tej grupy kluczowy jest niski próg wejścia — możliwość przeglądania katalogu bez rejestracji — oraz czytelne filtry i widoki (lista, mapa, kalendarz). Segment ten odpowiada w dużej mierze użytkownikowi anonimowemu (gościowi) w warstwie aplikacji.

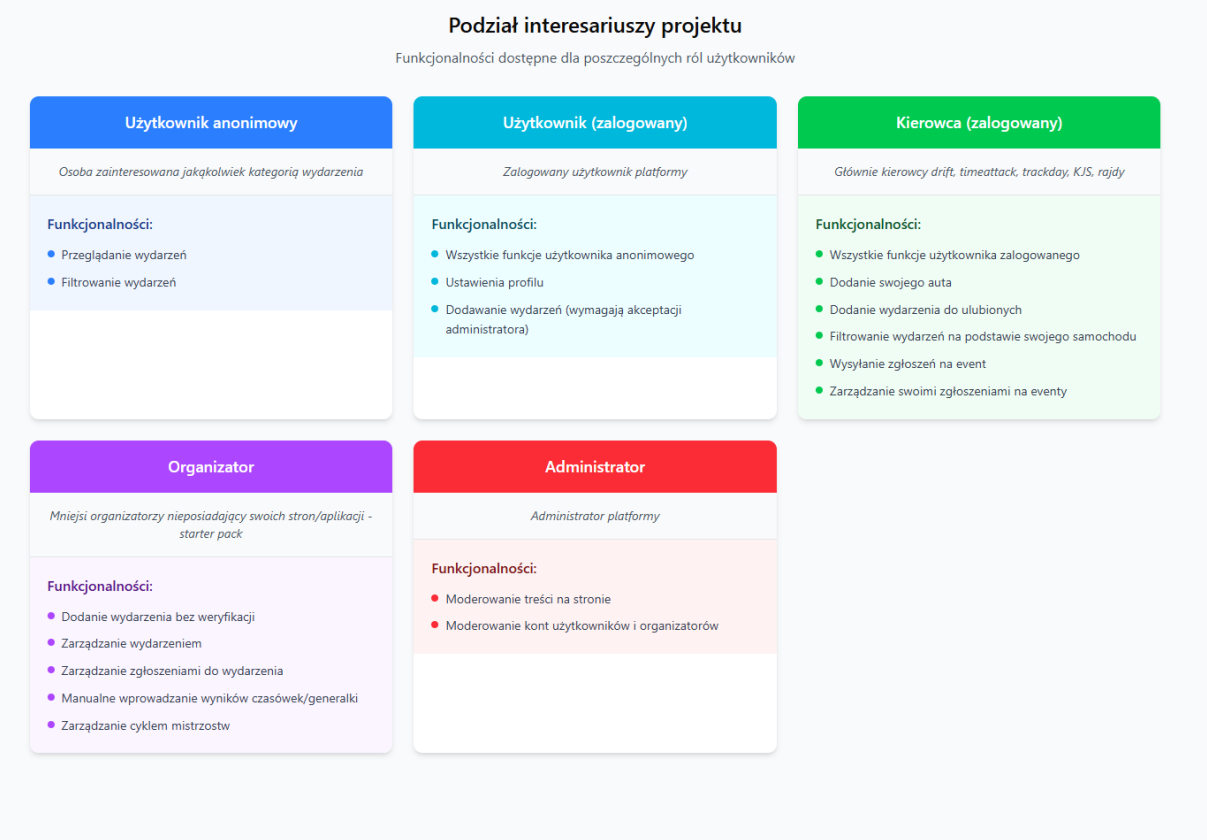
Uczestnicy wydarzeń (kierowcy) stanowią segment transakcyjny: potrzebują nie tylko znaleźć start, lecz także ocenić zgodność wymagań wydarzenia z posiadanym pojazdem, złożyć

zgłoszenie i śledzić jego status. W systemie segment ten jest obsługiwany przez konta użytkowników z dostępem do garażu i zgłoszeń. Hipoteza koncepcyjna KMR wiąże ten segment z rynkiem polskim i orientacyjnym przedziałem 16–40 lat; brakuje jednak wyników badań potwierdzających ten rozkład, dlatego nie jest on traktowany jako ustalenie empiryczne.

Organizatorzy wydarzeń dostarczają podaż treści i procesów zapisów. Ich potrzebą jest skuteczny kanał dotarcia do uczestników oraz narzędzie do publikacji imprezy i obsługi listy zgłoszeń (w tym komunikacji statusów). W praktyce produktowej dostęp organizatora wymaga weryfikacji wniosku przez operatora platformy, co stanowi mechanizm jakościowy, a nie osobny segment rynku.

Administrator nie jest segmentem rynkowym, lecz *rolą operacyjną* odpowiedzialną za funkcjonowanie platformy: moderację wydarzeń, rozstrzyganie wniosków o uprawnienia organizatora oraz nadzór nad jakością katalogu. W analizie biznesowej administrator należy do strony operatorskiej, a nie do grup klientów, do których kierowana jest propozycja wartości.

Wczesny, koncepcyjny podział funkcjonalności według poziomów dostępu (materiał projektowy zespołu) przedstawia rys. 4.1. Należy go interpretować jako mapę intencji produktowej; szczegółowe mapowanie na wymagania i projekt systemu opisano w rozdziałach 7–8.



Rysunek 4.1: Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny).

4.2 Interesariusze systemu

Analiza udziałowców obejmuje zarówno klientów platformy, jak i otoczenie opisane w dokumentacji SAI (m.in. zespół wytwórczy, media, właściciele obiektów, służby zabezpieczające imprezy). Poniżej przedstawiono karty kluczowych udziałowców w formacie środowiska `stakeholder` szablonu PJATK. W stosunku do wcześniejszej wersji dodano kartę gościa/kibica oraz właściciela toru / partnera obiektu, ponieważ obie grupy mają potrzeby odrębne od kierowcy i organizatora. Odwołania do wymagań wskazują rozdziały 7–8.

KARTA UDZIAŁOWCA	
Identyfikator:	U-00
Nazwa:	Gość / kibic (użytkownik anonimowy)
Opis:	Odbiorca informacji przeglądający publiczny katalog wydarzeń bez zakładania konta.
Typ udziałowca:	Użytkownik końcowy / odbiorca treści
Punkt widzenia:	Chce szybko znaleźć aktualne wydarzenia, lokalizacje i terminy bez bariery rejestracji.
Ograniczenia:	Bez konta nie może złożyć zgłoszenia ani prowadzić garażu; zależny od kompletności i aktualności katalogu publikowanego przez organizatorów.
Wymagania:	Publiczny katalog i filtry — rozdz. 7; widoki lista/mapa/kalendarz — rozdz. 11.

Tabela 4.1: Karta udziałowca: Gość / kibic

KARTA UDZIAŁOWCA	
Identyfikator:	U-01
Nazwa:	Kierowca
Opis:	Użytkownik z kontem, który utrzymuje garaż pojazdów i składa zgłoszenia na wydarzenia motorsportowe.
Typ udziałowca:	Użytkownik końcowy / klient platformy
Punkt widzenia:	Chce szybko znaleźć starty, dopasować pojazd do wymagań wydarzenia i śledzić status zgłoszenia w jednym miejscu.
Ograniczenia:	Korzystanie z garażu i zapisów wymaga rejestracji oraz podania danych niezbędnych do obsługi zgłoszenia; skuteczność zapisu zależy od odpowiedzi organizatora i kompletności wymagań wydarzenia.
Wymagania:	Wymagania funkcjonalne katalogu, garażu i zgłoszeń — rozdz. 7; panel użytkownika — rozdz. 11.

Tabela 4.2: Karta udziałowca: Kierowca

KARTA UDZIAŁOWCA	
Identyfikator:	U-02
Nazwa:	Organizator
Opis:	Podmiot publikujący wydarzenia i zarządzający listą zgłoszeń uczestników.
Typ udziałowca:	Dostawca treści / partner biznesowy
Punkt widzenia:	Potrzebuje skutecznego kanału dotarcia do uczestników oraz narzędzia do zarządzania wydarzeniami, weryfikacji zgłoszeń i komunikacji statusów.
Ograniczenia:	Publikacja wymaga pozytywnej weryfikacji uprawnień; rozliczanie wpisowego odbywa się poza platformą, więc organizator nadal prowadzi proces płatności we własnym zakresie; wartość panelu zależy od liczby aktywnych kierowców.
Wymagania:	Panel organizatora i kontrola dostępu — rozdz. 7, 8; realizacja — rozdz. 10.

Tabela 4.3: Karta udziałowca: Organizator

KARTA UDZIAŁOWCA	
Identyfikator:	U-03
Nazwa:	Administrator (rola operacyjna)
Opis:	Operator platformy moderujący wydarzenia, role kont oraz wnioski o uprawnienia organizatora.
Typ udziałowca:	Właściciel / operator systemu
Punkt widzenia:	Zapewnia jakość katalogu, spójność uprawnień i bezpieczeństwo działania platformy.
Ograniczenia:	Skuteczność moderacji zależy od dostępności operatora i jakości zgłoszeń od użytkowników; przy wzroście skali rośnie nakład pracy na weryfikację treści i ról.
Wymagania:	Panel administracyjny oraz bezpieczeństwo dostępu — rozdz. 13; testy kontroli dostępu — rozdz. 14.

Tabela 4.4: Karta udziałowca: Administrator

4.3 Propozycja wartości systemu

Propozycja wartości RacePortal wynika z diagnozy rozproszenia informacji o wydarzeniach motorsportowych w Polsce. Dotychczasowe praktyki — grupy w mediach społecznościowych, witryny poszczególnych torów oraz statyczne kalendarze — nie zapewniają jednolitego procesu wyszukiwania imprez, porównania wymagań ani obsługi statusu zapisu. RacePortal odpowiada na ten deficyt jako **centralny katalog** oraz **warstwa operacyjna zapisów**, z narzędziami dopasowanymi do potrzeb kibiców, kierowców i organizatorów.

Dla kierowców wartość koncentruje się na: przejrzystym katalogu z widokami lista, mapa i kalendarz oraz wspólnymi filtrami; wirtualnym garażu pojazdów; wsparciu dopasowania pojazdu do wymagań wydarzenia; złożeniu zgłoszenia i otrzymywaniu informacji o statusie.

KARTA UDZIAŁOWCA	
Identyfikator:	U-04
Nazwa:	Właściciel toru / partner obiektu
Opis:	Podmiot dysponujący infrastrukturą (tor, obiekt) wykorzystywaną przy organizacji wydarzeń publikowanych na platformie.
Typ udziałowca:	Partner pośredni / interesariusz otoczenia
Punkt widzenia:	Jest zainteresowany widocznością obiektu oraz poprawnością informacji o lokalizacji wydarzeń odbywających się na jego terenie.
Ograniczenia:	Nie zarządza bezpośrednio zapisami uczestników w MVP; wpływ na treść katalogu jest pośredni (poprzez organizatorów lub kontakt z operatorem); nie posiada dedykowanego panelu partnerskiego.
Wymagania:	Katalog lokalizacji i dane wydarzeń — rozdz. 7, 8.

Tabela 4.5: Karta udziałowca: Właściciel toru / partner obiektu

Dla organizatorów wartość obejmuje publikację wydarzenia, zarządzanie listą zgłoszeń oraz komunikację statusów bez konieczności prowadzenia całego procesu wyłącznie w arkuszach i wiadomościach prywatnych. Dla odbiorców informacji wartość sprowadza się do bezpłatnego dostępu do terminów i lokalizacji, w tym do widoku mapy oraz wskazówek dojazdu.

Wyróżnikiem względem prostych kalendarzy branżowych jest integracja łańcucha *znalezienie wydarzenia → dopasowanie pojazdu → zgłoszenie → status*, a nie wyłącznie publikacja dat. Najistotniejszym ryzykiem biznesowym pozostaje odmowa migracji społeczności z grup społecznościowych do scentralizowanego narzędzia; bez aktualnej podaży wydarzeń katalog traci wartość dla kierowców (klasyczny problem platformy dwustronnej).

Tabela 4.6 przedstawia porównanie *modeli działania*, a nie empiryczne badanie konkurencji. Ilustruje, na czym polega zakładana przewaga RacePortal względem typowego sposobu organizacji informacji i zapisów w grupach społecznościowych.

Tabela 4.6: Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne).

Kryterium	**Grupy społecznościowe**	**RacePortal (model docelowy)**
Wyszukiwanie wydarzeń	Zależne od postów i wyszukiwarki grupy	Ustrukturyzowane filtry i wspólny katalog
Porównanie wydarzeń	Ręczne, między rozproszonymi źródłami	Wspólny katalog z ujednoliconym opisem
Dopasowanie pojazdu	Weryfikacja po stronie uczestnika	Wsparcie dopasowania względem wymagań wydarzenia
Zgłoszenie udziału	Zależne od praktyki danego organizatora	Ujednolicony proces zgłoszenia
Status zgłoszenia	Różne kanały komunikacji	Centralny podgląd statusu i powiadomienia

4.4 Kanały komunikacji i dystrybucji

Głównym kanałem dostępu do produktu jest aplikacja webowa (`https://raceportal.pl`), umożliwiająca zarówno przeglądanie katalogu, jak i obsługę konta, garażu, zgłoszeń oraz paneli organizatora i administratora. Uzupełnieniem jest mobilna aplikacja-wizytówka, służąca do odkrywania wydarzeń i przekierowania zapisu do wersji webowej. Takie rozdzielenie obniża koszt wejścia informacyjnego, a jednocześnie koncentruje procesy wymagające konta w jednym miejscu.

Z perspektywy pozyskiwania użytkowników kluczowe są kanały organiczne i partnerskie, przewidziane także w materiałach BMC: obecność w mediach społecznościowych społeczności motorsportowej, bezpośrednie kontakty z klubami i organizatorami, promocja podczas imprez torowych oraz współpraca z właścicielami obiektów. W fazie MVP działania te mają charakter organizacyjny — ich celem jest zbudowanie podaży aktualnych wydarzeń i pierwszej grupy aktywnych kierowców, bez których katalog nie generuje efektu sieciowego.

Kanałem utrzymania relacji z użytkownikiem po rejestracji lub zgłoszeniu są powiadomienia e-mail (aktywacja konta, statusy zgłoszeń, istotne zmiany wydarzenia). Pełnią one funkcję komunikacyjną i operacyjną, a nie marketingową: informują o stanach procesu, zamiast prowadzić kampanie sprzedażowe. Szczegóły techniczne dostarczania poczty opisano w rozdziałach implementacyjnych.

4.5 Kluczowe zasoby i działania

Do zasobów kluczowych należą zarówno zasoby technologiczne, jak i biznesowo-operacyjne. Po stronie technologicznej są to: działająca aplikacja webowa i mobilna wizytówka, backend z trwałą bazą wydarzeń oraz konteneryzowane środowisko uruchomieniowe umożliwiające powtarzalne wdrożenie. Zasobem produktowym jest także katalog torów i treści wydarzeń — bez aktualnych danych platforma traci wartość niezależnie od jakości oprogramowania.

Po stronie biznesowej i operacyjnej kluczowi są: zespół projektowy utrzymujący rozwój i moderację; relacje z organizatorami i obiektami (źródło podaży wydarzeń); domena oraz niskokosztowa infrastruktura hostingowa; a także dokumentacja procesów i scenariusze prezentacji ułatwiające weryfikację produktu. W ujęciu BMC zasoby te są komplementarne: technologia bez relacji z organizatorami nie zapełni katalogu, a relacje bez sprawnego narzędzia nie utrzymają powtarzalnego procesu zapisów.

Działania kluczowe obejmują: rozwój i utrzymanie platformy; zapewnienie uruchamialności środowiska (Compose lokalnie i na VM); moderację treści oraz weryfikację organizatorów; utrzymanie mechanizmów bezpieczeństwa (JWT [6], RBAC, walidacja wejścia, kierunek zgodności z OWASP ASVS Level 1 [7]); utrzymanie dostarczalności powiadomień e-mail; a także dokumentowanie zmian i weryfikację testami automatycznymi. W ujęciu BMC działania mar-

ketingowe i budowa społeczności są uznane za konieczne dla przełamania ryzyka niskiego zainteresowania, lecz pozostają poza zautomatyzowanym zakresem oprogramowania MVP.

4.6 Model działania platformy

RacePortal funkcjonuje jako platforma dwustronna: organizatorzy dostarczają podaż wydarzeń, a kierowcy generują popyt na starty. Odbiorcy informacji korzystają z efektu sieciowego katalogu bez bariery rejestracji, co obniża koszt wejścia. Administrator — jako rola operacyjna — pełni funkcję gwaranta jakości poprzez akceptację wydarzeń i uprawnień.

W fazie MVP platforma **nie** rozlicza płatności za udział. Dla wydarzeń płatnych system obsługuje statusy zgłoszenia oraz dołączenie dowodu wpłaty, natomiast przelew lub inna forma rozliczenia odbywa się poza platformą, według zasad organizatora. Model darmowego dostępu obejmuje przeglądanie katalogu oraz korzystanie z narzędzi kierowcy i panelu organizatora. Na tym etapie weryfikacja wartości produktu opiera się na użyteczności operacyjnej, a nie na monetyzacji.

Z punktu widzenia utrzymania platformy materiały BMC wskazują potencjalne, *przyszłe* strumienie przychodów: płatną promocję / wyróżnienie wydarzeń, reklamy powiązane z motorsportem oraz — w dłuższym horyzoncie — prowizje od zapisów lub biletów. Równolegle występują koszty stałe i zmienne: hosting i domena, obsługa poczty, rozwój oprogramowania, moderacja treści oraz ewentualne działania promocyjne. W obecnym etapie projekt ma charakter dydaktyczno-inżynierski i opiera się na infrastrukturze niskokosztowej oraz pracy zespołu; nie przedstawia się tu prognoz finansowych liczbowych, ponieważ nie zebrano danych rynkowych wystarczających do wiarygodnej estymacji.

Praktycznym sposobem sprawdzenia zainteresowania odpłatnymi funkcjami byłoby — po okresie bezpłatnego użytkowania — przeprowadzenie rozmów lub prostych testów oferty z aktywnymi organizatorami (np. wyróżnienie wydarzenia, ekspozycja w katalogu), zgodnie z kierunkiem walidacji założonym w dokumencie *Interview Validation*. Dopiero potwierdzenie gotowości do zapłaty za konkretną korzyść uzasadniałoby przejście od modelu darmowego MVP do wariantu monetyzacji.

4.7 Realistyczność wdrożenia i dalsze perspektywy

Realistyczność techniczna wdrożenia została potwierdzona uruchomieniem środowiska lokalnego oraz publikacją działającej instancji produkcyjnej w modelu niskokosztowym (maszyna wirtualna w chmurze z udostępnieniem poprzez tunel HTTPS). Szczegóły architektury fizycznej zawarto w rozdz. 8.3; w niniejszym podrozdziale istotniejsza jest ocena realności *biznesowej* utrzymania produktu.

Kluczowym warunkiem sukcesu nie jest sama dostępność serwisu, lecz pozyskanie organizatorów publikujących aktualne wydarzenia oraz utrzymanie kompletności katalogu. Bez regularnej podaży treści platforma nie różni się istotnie od statycznego kalendarza i nie motywuje kierowców do rejestracji. Dlatego realistyczny scenariusz rozwoju zakłada: bezpośrednie kontakty z organizatorami i obiektami, prosty proces publikacji, moderację zapewniającą wiarygodność oraz mechanizmy powiadomień ograniczające ryzyko dezaktualizacji informacji u uczestników.

Założona w DZW skala pilotażowa (rzędu tysięcy użytkowników miesięcznie przy profilu głównie odczytowym) jest spójna z niszowym charakterem amatorskiego motorsportu w Polsce i z ograniczonym budżetem operacyjnym projektu. Formalny test obciążenia przy ok. 10 000 rekordach / 50 RPS nie został domknięty jako artefakt odbiorczy KP/DZW; wykonano natomiast krótkie pomiary Grafana k6 na Oracle VM (25.10.2026), opisane w `docs/testy/wyniki/k6-vm-rps.md` [8] i omówione w rozdziałach testowania oraz podsumowania.

Dalsze perspektywy obejmują m.in. integrację płatności online, rozbudowę powiadomień, publikację aplikacji mobilnej w sklepach, eksport list startowych oraz ewentualną monetyzację oznaczonych treści promocyjnych. Elementy te stanowią roadmapę produktową, a nie stan bieżącej realizacji.

4.8 Kontekst prawny i etyczny

System przetwarza dane niezbędne do obsługi konta i procesów biznesowych: identyfikator/nazwę użytkownika, adres e-mail, role, dane pojazdów w garażu, zgłoszenia oraz dane wydarzeń wprowadzane przez organizatorów. Sesja webowa opiera się o tokeny JWT w ciasteczkach httpOnly (`rp_at`, `rp_rt`) wysyłanych z `credentials: "include"`; starszy JWT w `localStorage` jest usuwany przy starcie aplikacji. W interfejsie udostępniono politykę prywatności (`/privacy`) oraz regulamin (`/terms`); rejestracja wymaga akceptacji obu dokumentów. Przy tworzeniu wydarzenia organizator potwierdza przetwarzanie danych zgodnie z polityką prywatności (RODO) wyłącznie w celu publikacji i obsługi zgłoszeń. Kanałem kontaktowym w sprawach ochrony danych jest adres `rodo@raceportal.pl`. Użytkownik może zamknąć własne konto w ustawieniach (fraza potwierdzająca po stronie API).

Należy rozróżnić *obowiązki już wynikające z przetwarzania danych* od *uszprawnień organizacyjnych* przewidzianych na późniejszy etap skalowania (np. bardziej rozbudowane procedury wewnętrzne, sformalizowana polityka retencji, ewentualna ocena skutków dla ochrony danych). Ocena skutków dla ochrony danych (DPIA) nie jest automatycznie wymagana dla każdego systemu — jej konieczność zależy od charakteru i ryzyka przetwarzania. W pracy nie rozstrzyga się, czy dla RacePortal DPIA jest obligatoryjna; wskazuje się jedynie, że przy wzroście skali i zakresu danych decyzja ta powinna zostać ponownie oceniona.

W warstwie odpowiedzialności za treści: organizator odpowiada za merytoryczną treść publikowanych wydarzeń w ramach regulaminu, jednak nie wyłącza to obowiązków operatora platformy w zakresie udostępniania narzędzia, moderacji oraz reagowania na zgłoszenia nadużyć. Model przyjęty w projekcie łączy weryfikację organizatorów i moderację wydarzeń przez administratora z zasadami etycznymi opisanymi w dokumentacji SAI (transparentność prezentacji, unikanie mechanizmów uzależniających, przeciwdziałanie publikacji niebezpiecznych lub nielegalnych aktywności). Ostateczne decyzje moderacyjne pozostają po stronie człowieka.

Rozdział 5

Analiza SWOT

Analiza SWOT (ang. *Strengths*, *Weaknesses*, *Opportunities*, *Threats*) służy w tej pracy do uporządkowanej oceny RacePortal jako produktu MVP: co stanowi przewagę rozwiązania względem rozproszonego rynku informacji o startach, gdzie leżą ograniczenia obecnej wersji, jakie kierunki rozwoju są realne oraz jakie czynniki zewnętrzne mogą utrudnić utrzymanie lub skalowanie systemu. Ocena odnosi się do **stanu zaimplementowanego** (web, API, mobile, wdrożenie produkcyjne), a nie wyłącznie do założeń koncepcyjnych.

5.1 Mocne strony (Strengths)

Mocne strony wynikają przede wszystkim z domknięcia spójnego happy-path dla wszystkich ról oraz z uruchamialności środowiska demonstracyjnego i produkcyjnego.

- Pełny przepływ ról: gość, kierowca (`USER`), organizator (`ORGANIZER`) i administrator (`ADMIN`), z kontrolą dostępu RBAC oraz uwierzytelnianiem JWT (w tym opcjonalne logowanie Google).
- Publiczny katalog wydarzeń z trzema widokami na wspólnych filtrach (lista / mapa / kalendarz), detal wydarzenia, wyznaczanie trasy (OSRM) oraz archiwum.
- Panel organizatora jako workspace wydarzenia: zarządzanie zgłoszeniami (statusy, klasy zawodów, eksport CSV), wyniki z publikacją, komunikacja masowa oraz KPI wyświetleń.
- Widok publiczny listy startowej i opublikowanych wyników na stronie wydarzenia.
- Dedykowany backoffice administratora (`bo.raceportal.pl`) z moderacją i eksportami, oddzielony od apexu prezentacyjnego.
- Konteneryzacja (Docker Compose), seed danych demo, powiadomienia e-mail (Mailbux) oraz wdrożenie produkcyjne (Oracle Always Free + Cloudflare Tunnel, TLS po stronie Cloudflare).

- Warstwa web jako PWA; wspólne typy API (`packages/api-types`) ograniczające dryf web $\leftrightarrow$ mobile.
- Testy automatyczne (integracyjne API, E2E Playwright, unit mobile / api-types) oraz pomiary obciążenia k6 na VM produkcyjnej (wąskie gardło: endpointy z bazą danych).

5.2 Słabe strony (Weaknesses)

Słabe strony to świadome lub nierozwiązane ograniczenia MVP, które obniżają kompletność produktową albo utrudniają odbiór jakościowy bez dodatkowych prac.

- Brak natywnej bramki płatności (PSP) — obowiązuje model „zapłać później" z dowodem przelewu (upload pliku / URL), a nie checkout online; upload jest w webie/API, nie w mobilce.
- Aplikacja mobilna Expo pozostaje wizytówką katalogu: bez logowania, garażu i pełnej parity z webem; brak publikacji w sklepach App Store / Google Play.
- Zależność od publicznych serwisów mapy i routingu (CARTO / OSRM; opcjonalnie Google Maps przy kluczu API) — awaria lub limity zewnętrzne pogarszają UX lokalizacji.
- Częściowe rozjazdy względem wewnętrznej Specyfikacji Formularzy (udokumentowane w rejestrze luk).
- Nie domknięto pierwotnego kryterium jakości „ok. 10 000 rekordów w katalogu" jako osobnego scenariusza obciążeniowego; pomiary k6 dotyczyły przepustowości RPS na istniejących danych, a nie zapełnienia katalogu do progu 10k.
- Ograniczenia hostingu Always Free (m.in. współdzielone zasoby CPU przy testach obciążeniowych) oraz dalszy hardening produkcyjny (np. pełne domknięcie kontroli dostępu do zasobów uploadów) pozostają w zakresie prac jakościowych, nie jako osobna funkcja produktowa MVP.

5.3 Szanse (Opportunities)

Szanse wynikają z rozpoznawalnego problemu rynkowego (rozproszone informacje o startach) oraz z już istniejącego rdzenia platformy, który można rozszerzać przyrostowo.

- Integracja PSP (np. Przelewy24 / Stripe) oraz fakturowanie / zwroty — domknięcie ścieżki płatnej bez ręcznej weryfikacji dowodu.

- Waitlista miejsc, watchlista wydarzeń oraz powiadomienia push / iCal — zwiększenie retencji kierowców i redukcja kanałów społecznościowych jako jedynego źródła aktualności.
- Rozszerzenie eksportów i odprawy (PDF listy startowej, kod QR zgłoszenia) na bazie już działającego CSV i publicznej listy startowej.
- Budowa społeczności wokół torów i organizatorów regionalnych (featured events, partnerzy, współorganizatorzy).
- Stopniowe zbliżanie mobile do weba (konto, garaż) albo świadome utrzymanie wizytówki z silniejszą konwersją PWA / deep-link.

5.4 Zagrożenia (Threats)

Zagrożenia mają charakter zewnętrzny: konkurencja zachowań użytkowników, zależność od infrastruktury i dostawców oraz obowiązki prawne przy skali produkcyjnej.

- Konkurencja ze strony grup społecznościowych, komunikatorów oraz dedykowanych systemów zapisów organizatorów — niska bariera „zostania przy Facebooku / Discordzie".
- Zmiany limitów, cenników lub dostępności publicznych API mapy i routingu (oraz polityk OAuth Google).
- Wymogi RODO i odpowiedzialność za dane osobowe przy wzroście skali (retencja, DPIA, obsługa żądań podmiotów danych) — UI zawiera politykę prywatności i notę RODO, ale pełna zgodność procesowa wykracza poza demonstracyjne MVP.
- Ryzyko nadużyć powierzchni ataku typowej dla serwisów publicznych (spam kont / zgłoszeń, scraping katalogu, niewłaściwe użycie uploadów) mimo wdrożonych kontroli JWT/RBAC i auditów bezpieczeństwa.
- Uzależnienie od limitu darmowej / niskokosztowej infrastruktury chmurowej (Oracle Always Free, Cloudflare Tunnel) przy niespodziewanym wzroście ruchu.

5.5 Podsumowanie analizy SWOT

RacePortal ma silny rdzeń funkcjonalny i techniczny jak na MVP akademickie: spójne role, katalog, zapisy, narzędzia organizatora i administratora, testy oraz działające wdrożenie. Główne słabości dotyczą świadomie odłożonych elementów produktowych (PSP, pełne mobile) oraz domknięcia wybranych kryteriów jakościowych (scenariusz 10k rekordów). Szanse rozwoju

są naturalną kontynuacją istniejącego API i paneli; zagrożenia wymagają przede wszystkim dyscypliny bezpieczeństwa, prywatności i niezależności od pojedynczych usług zewnętrznych. Wnioski te są spójne z ograniczeniami i perspektywami opisanymi w rozdziale 18.

Rozdział 6

Planowanie projektu

Ten rozdział opisuje planowanie RacePortal: cele, zakres MVP, sposób pracy, zespół, harmonogram, infrastrukturę oraz ryzyka. Oddzielam tu dokumenty formalne — Kartę projektu (KP), Dokument Założeń Wstępnych (DZW), Specyfikację Wymagań Systemowych (SWS) oraz historyczny Harmonogram przedmiotu PRO — od tego, co faktycznie powstało w kodzie, Docker Compose, plikach `docs/MVP.md`, chronologii `docs/changes.md`, decyzjach wdrożeniowych i pomiarach k6. Daty publikacji dokumentów i daty wpisów w chronologii to dwie różne warstwy: przebieg prac nie zastępuje oficjalnych kamieni milowych karty projektu, a karty formalne nie zastępują rzeczywistego stanu systemu.

6.1 Cele projektu

Głównym celem RacePortal, zgodnie z KP (data ustalenia tematu 01.10.2025), jest zbudowanie jednego miejsca z informacjami o wydarzeniach motorsportowych w Polsce — oficjalnych i amatorskich (m.in. drift, time attack, track day, KJS) — oraz narzędzi dla organizatorów do publikacji imprez i obsługi zapisów. Opiekunem (promotorem) projektu w KP i w pracy dyplomowej jest Marek Bednarczyk. KP rozwija cel główny w czterech skutkach: zebranie danych w jednym miejscu (odejście od chaosu grup społecznościowych i nieaktualnych stron), wsparcie mniejszych organizatorów, udogodnienia dla kierowców (profil, wirtualny garaż, szybkie zgłoszenia) oraz popularyzacja motorsportu przez kalendarz i mapę.

DZW — którego historia wersji w nagłówku sięga m.in. v1.4 z 26.05.2026 — doprecyzowuje cele systemu: katalog z listą i kalendarzem, filtrowanie (kategoria, tor, województwo, data, nazwa), szczegółowy podgląd wydarzeń, panel organizatora, konto kierowcy z garażem i zgłoszeniami oraz fundament pod dalszy rozwój. SWS formalizuje te cele jako podstawę implementacji, testów i oceny produktu akademickiego. W warstwie problemowej DZW wskazuje rozproszenie informacji o zawodach oraz brak jednego kanału dla organizatorów do publikacji i zarządzania wydarzeniem — to nadal punkt wyjścia przy planowaniu zakresu.

W pracy inżynierskiej celem praktycznym jest działające MVP demonstracyjne: katalog

i zapisy z rolami użytkowników, mapa i trasa, powiadomienia e-mail, ścieżka mobilna, konteneryzacja oraz dokumentacja i testy umożliwiające odbiór lub prezentację. Miary sukcesu z KP obejmują m.in. stabilne uruchomienie w Dockerze, cele wydajnościowe API dla żądań listujących, zgodność z OWASP ASVS Level 1 i RODO, pozytywne testy manualne kluczowych punktów zakresu oraz przygotowanie systemu na obsługę rzędu 1–5 tys. użytkowników miesięcznie (skala pilotowa z DZW/KP). Te miary wyznaczają kierunek planowania jakości; ich sprawdzenie w praktyce należy do rozdziałów realizacji i testowania, a nie zastępuje planu.

6.2 Zakres projektu

Zakres MVP jest spójny między KP, DZW i `docs/MVP.md` jako lista piętnastu funkcji: kalendarz i lista wydarzeń; filtry oraz wyszukiwarka; szczegóły wydarzenia; panel administratora; konto kierowcy; garaż; system zgłoszeń; konto organizatora; narzędzia organizatora; baza danych wydarzeń; powiadomienia mailowe; mapa wydarzeń; integracja mapowa trasy (w KP i DZW: Google Maps API); archiwum; aplikacja mobilna (w KP: PWA / mobile-first).

Poza zakresem MVP — świadomie — pozostają m.in. integracje z kalendarzami zewnętrznymi, powiadomienia push poza mailami, automatyczna integracja z mediami społecznościowymi oraz live timing / pomiary czasów. KP wymienia je jako możliwości dalszej rozbudowy; w planowaniu nie traktuje się ich jako zobowiązań odbiorczych.

Warunki odbioru przyjęte w DZW i powtórzone w `docs/MVP.md` wymagają, by funkcje 1–11 były dostępne i przetestowane (manualnie oraz podstawowymi testami e2e), by UX zachowywał podejście mobile-first oraz by zakres był spójny z pitchiem i prezentacją. Jakość planowana obejmuje codzienne kopie zapasowe i kontrole stanu (health checks), Docker, paginację, indeksy i cache, czas odpowiedzi P95 nie większy niż 500–700 ms dla list do 50 rekordów, test obciążenia przy ok. 10 000 rekordów i 50 RPS, OWASP ASVS Level 1, RBAC, HTTPS z nagłówkami bezpieczeństwa, minimalizację danych (RODO) oraz przenośność środowiska Docker.

Według synchronizacji `docs/MVP.md` z 28.10.2026 funkcje 1–12 oraz 14–15 są domknięte funkcjonalnie (z lukami względem Specyfikacji Formularzy przy koncie kierowcy), a funkcja 13 — częściowo: trasa działa przez publiczne OSRM, a Google Maps pozostaje jedynie opcją konfiguracyjną. Ścieżkę mobilną zrealizowano jako wizytówkę Expo oraz webową PWA, a nie jako pełną wersję natywną ani publikację w sklepach — to świadoma adaptacja wobec sformułowania KP „PWA / Mobile-first”. Upload plików (obrazy JPG/PNG/WebP oraz PDF) istnieje w API (`/api/uploads/...`) i jest odnotowany w MVP jako domknięty 23.10.2026; nadal pozostają ograniczenia (m.in. brak pola zdjęcia w mobilce oraz niekompletna zgodność formularzy ze Specyfikacją). Elementy zrealizowane ponad pierwotną listę piętnastu funkcji (m.in. motywy, i18n, regulaminy PDF, tagi opon) odróżnia się w opisie od pierwotnego MVP,

żeby nie zacierać granicy zakresu.

Tabela 6.1: Zakres MVP — funkcje z KP/DZW oraz stan według `docs/MVP.md` (28.10.2026).

Nr	Funkcja	Stan realizacji
1	Kalendarz / lista wydarzeń	Zrealizowane: lista z API, paginacja, UI `/wydarzenia`
2	Filtry i wyszukiwarka	Zrealizowane: m.in. `q`, kategoria, miasto, filtr płatności
3	Szczegóły wydarzenia	Zrealizowane: detal, zapis, trasa, regulaminy PDF
4	Panel administratora	Zrealizowane: użytkownicy, eventy, wnioski org., błędy
5	Konto kierowcy	Częściowo: JWT, dashboard, weryfikacja e-mail; luki vs Spec
6	Garaż	Zrealizowane: CRUD pojazdów; zdjęcie przy dodaniu; tagi opon
7	System zgłoszeń	Zrealizowane: statusy, maile; soft termin dowodu wpłaty
8	Konto organizatora	Zrealizowane: wniosek o rolę + akceptacja admina
9	Narzędzia organizatora	Zrealizowane: tworzenie/edycja eventów, plakat, PDF
10	Baza danych wydarzeń	Zrealizowane: MySQL + JPA; seed torów PL
11	Powiadomienia mailowe	Zrealizowane: SMTP Mailbux
12	Mapa wydarzeń	Zrealizowane: Leaflet + OpenStreetMap
13	Trasa (plan: Google Maps)	Częściowo: OSRM domyślnie; Google Maps tylko opcjonalnie
14	Archiwum	Zrealizowane: archiwum + auto-archiwizacja
15	Aplikacja mobilna	Zrealizowane: Expo jako wizytówka + web PWA

6.3 Metodyka pracy

W historycznej dokumentacji planistycznej przedmiotu PRO (*Harmonogram* v02, folder „stare wersje”; sporządzenie 04–06.11.2025) przyjęto cykl przyrostowy z prototypami: każdy etap obejmuje planowanie, implementację i test wybranego modułu (np. kalendarz, panel administratora, mapy). Etapy nazwane w Harmonogramie to: analiza i planowanie, projektowanie, implementacja backendu, implementacja frontendu i integracje, testy oraz dokumentacja, a następnie prezentacja i ewaluacja MVP. Jako narzędzia planowania wymieniono m.in. GitHub Projects oraz — w części zasobowej — JIRA lub Monday.com; nie wynika z tego, by sprinty Scrum lub Kanban z datami były formalnie egzekwowane, więc nie wprowadza się tu wymyślonych tablic ani velocity. Harmonogram PRO traktuje się wyłącznie jako materiał historyczny przedmiotowy, nie jako obowiązującą kartę dyplomu; opiekun przedmiotu PRO w tym dokumencie (Marta Łabuda) nie zastępuje promotora pracy dyplomowej (Marek Bednarczyk).

W praktyce w repozytorium widać iteracyjne dostarczanie w monorepo: wspólny kontrakt API, bieżąca dokumentacja zmian (układ „było → jest → dlaczego” w `docs/changes.md`), synchronizacja stanu względem MVP oraz decyzje infrastrukturalne (`docs/decyzje-wdrozeniowe.md`). Przegląd przed merge do `main` uzupełniają testy JUnit/MockMvc, Vitest i Playwright. Prototyp UI w Figma Make („Advanced UI/UX for RACEPORTAL”) był punktem startu frontendu; kolejne fale sprawdzeń UI i poprawek dokumentowano jako prace jakościowe, a nie jako formalny wykres Gantta. Zarządzanie jakością planowane w Harmonogramie PRO (code

review przed `main`, testy manualne i automatyczne, inspekcja dokumentacji) pozostaje spójne z praktyką repozytorium, choć konkretne narzędzia się zmieniały (m.in. Postman z planu historycznego ustąpił miejsca testom JUnit/MockMvc i Playwright).

6.4 Organizacja zespołu

Zespół jest pięcioosobowy (specjalizacja cyberbezpieczeństwo, tryb stacjonarny — KP): Natalia Otrombke, Oliwier Kasprowicz, Michał Gutowski, Miłosz Parkitny i Wojciech Wronisz. Opiekunem projektu w KP oraz w pracy dyplomowej jest Marek Bednarczyk.

Współpraca opiera się o wspólne repozytorium GitHub, gałąź `main` oraz dokumentację w `docs/` i `Dokumentacja/`. Jakość planowana obejmuje przegląd przed merge, testy manualne i automatyczne oraz bieżącą aktualizację dokumentów porównawczych (MVP, zgodność ze Specyfikacją Formularzy). Historyczny Harmonogram PRO przewidywał m.in. cotygodniowe statusy na Discordzie oraz prezentacje postępów zgodne z planem zajęć — te zapisy to kontekst organizacyjny przedmiotu, a nie formalny regulamin dyplomu.

Za formalny podział ról w tym rozdziale przyjmuje się zapis z DZW: kierownik projektu — Wojciech Wronisz; frontend — Natalia Otrombke i Oliwier Kasprowicz; backend — Miłosz Parkitny; fullstack / marketing — Michał Gutowski. Późniejsza praktyka opisana w `README.md` repozytorium różni się w szczegółach (kierownik: Michał Gutowski; backend: Michał Gutowski i Miłosz Parkitny; fullstack, testy i bezpieczeństwo: Wojciech Wronisz) — odnotowuje się ją jako zmianę organizacyjną w czasie, bez przypisywania tu autorstwa poszczególnych funkcji. Opis wkładu merytorycznego pozostaje w osobnym rozdziale pracy.

Tabela 6.2: Podział ról według DZW (formalny dla tego rozdziału) oraz opis w `README.md`.

Osoba	**Rola w DZW**	**Opis w** `README.md`
Wojciech Wronisz	Kierownik projektu	Fullstack, testy i bezpieczeństwo
Natalia Otrombke	Frontend	Frontend
Oliwier Kasprowicz	Frontend	Frontend
Miłosz Parkitny	Backend	Backend
Michał Gutowski	Fullstack / marketing	Kierownik; backend
Marek Bednarczyk	Opiekun (promotor)	Opiekun projektu

6.5 Harmonogram pracy

W KP data 31.01.2026 figuruje jako ograniczenie czasowe karty projektu. W tej pracy datę tę traktuje się jako termin dokumentacyjny karty (ograniczenie planistyczne KP), a **nie** jako kalendarzowy termin domknięcia gotowego i przetestowanego systemu. Przebieg implementacji — udokumentowany w `docs/changes.md` — wykracza poza tę datę karty i obejmuje m.in.

wdrożenie produkcyjne oraz domknięcia funkcji w październiku 2026; nie wprowadza się tu konkurencyjnej „daty ukończenia” spoza potwierdzonej interpretacji karty.

Planowanie etapowe z Harmonogramu PRO (analiza, projekt, backend, frontend z integracjami, testy, prezentacja MVP) stanowiło plan prac na zajęciach przedmiotu, a nie zamknięty formalny Gantt dyplomu. W Harmonogramie PRO podano również okno zajęć (ok. 15 tygodni od 6.10.2025 do 19.01.2026) oraz szacunek nakładu — te zapisy pozostają historią przedmiotową i nie są tu przepisywane jako obowiązujący harmonogram dyplomu z wymyślonymi datami kamieni milowych.

Przebieg prac — w odróżnieniu od kamieni milowych karty — dokumentuje plik `docs/changes.md`. Chronologia obejmuje m.in. przeniesienie prototypu z Figma Make do lokalnego monorepo, szkielet projektu Vite i Dockera, migrację API na Spring Boot z MySQL, budowę aplikacji mobilnej Expo jako wizytówki, iteracje testów i formularzy, a w październiku 2026 przenosiny produkcyjne na Oracle Always Free z Cloudflare Tunnel, SMTP Mailbux i webową PWA, następnie domknięcie uploadu plików oraz pomiary k6 na VM. Daty wpisów w `changes.md` są kotwicami przebiegu; nie zastępują oficjalnego terminu karty KP i nie tworzą formalnego wykresu Gantta.

Tabela 6.3: Etapy prac — plan z Harmonogramu PRO wobec charakteru przebiegu w repozytorium.

lp.	Etap (plan PRO)	Produkty planowane	Charakter w przebiegu
1	Analiza i planowanie	DZW, plan projektu	KP/DZW/SWS jako ramy; aktualizacje DZW w historii wersji
2	Projektowanie	SWS, makiety UI	Prototyp Figma Make; model danych i API w implementacji
3	Implementacja backendu	Moduły REST, baza	Spring Boot + MySQL (zamiast historycznego Postgres z Harmonogramu)
4	Frontend i integracje	Widoki React, mapy	Vite/React; Leaflet/OSRM; Expo wizytówka; PWA
5	Testy i dokumentacja	Raporty, README	JUnit/MockMvc, Vitest, Playwright; `docs/`
6	Prezentacja MVP	Demo odbiorcze	Compose lokalnie + prod Oracle/Tunnel; scenariusz prezentacji

6.6 Infrastruktura i środowisko pracy

Wizja konstrukcyjna DZW zakładała frontend React, backend Java Spring Boot (REST), bazę MySQL, integracje mapowe, GitHub, Docker, Maven oraz GitHub Projects. W realizacji lokalnej i produkcyjnej źródłem prawdy uruchomieniowej jest Docker Compose: kontenery `web`

(nginx z buildem Vite/React), `api` (Spring Boot), `mysql` (MySQL 8.4) oraz `backup` (cykliczny `mysqldump`). Aplikacja mobilna Expo działa poza Compose. Maile systemowe idą SMTP przez Mailbux (hasło wyłącznie w `.env`).

Wdrożenie produkcyjne zrealizowano na maszynie Oracle Always Free (ARM Ampere) z pełnym Compose. Publiczny dostęp zapewnia Cloudflare Tunnel do nginx na porcie `:8081`; domena `raceportal.pl` korzysta z TLS, HSTS i DNS po stronie Cloudflare. Role warstw są rozdzielone: frontend (statyczny SPA za nginx z proxy `/api`), API (Spring Boot) oraz baza (MySQL). Cloudflare pełni rolę tunelu i bramy TLS/DNS, a nie hostingu frontu na Pages — wcześniejsze warianty GCP/Pages pozostają wyłącznie historią decyzji, opisaną w `docs/decyzje-wdrozeniowe.md`.

Stack mapowy w kodzie to Leaflet z kafelkami OpenStreetMap po stronie weba oraz wyznaczanie trasy przez publiczne OSRM (proxy `/api/maps/route`). Jest to świadoma realizacja względem planu KP/DZW opartego o Google Maps API; klucz Google Maps może figurować w konfiguracji, lecz aktywna ścieżka routingu w serwisie map korzysta z OSRM.

Środowisko pracy obejmuje Docker Desktop, Node.js (web i mobile), JDK/Maven (API w kontenerze), Expo z symulatorem lub emulatorem, Playwright, GitHub oraz `cloudflared` na VM Oracle. Upload plików jest częścią API (katalog wolumenowy `UPLOAD_DIR` w Compose), z ograniczeniami odnotowanymi w MVP i Specyfikacji Formularzy — nie zastępuje pełnej zgodności ze wszystkimi polami formularzy ani równoważności funkcji w aplikacji mobilnej.

6.7 Analiza ryzyka projektowego

6.7.1 Analiza zagrożeń

Poniżej zebrano ryzyka istotne dla planowania i odbioru MVP, wraz ze sposobami ograniczenia ryzyka wynikającymi z dokumentacji i stanu systemu. Część zagrożeń planowano już w historycznym Harmonogramie PRO (opóźnienia, spójność kodu, integracja map); poniżej ujęto je łącznie z ryzykami ujawnionymi w przebiegu implementacji i porównaniu plan–stan.

W zakresie wydajności empiryczne pomiary Grafana k6 na Oracle VM (25.10.2026; raport `docs/testy/wyniki/k6-vm-rps.md` [8]) wskazują m.in. komfortowy zakres ok. 150–200 RPS dla paginowanej listy wydarzeń (`GET /api/events`) — wąskim gardłem są endpointy z MySQL, a nie nginx ani lekki health API. Pomiar na współdzielonej VM (k6 lokalnie ze stackiem) zaniża wynik względem zewnętrznego generatora i nie zastępuje formalnego testu 10 000 rekordów / 50 RPS; szczegółowa analiza należy do rozdziału testowania.

Planowanie RacePortal łączy zatem formalne ramy KP/DZW z iteracyjnym przebiegiem implementacji. Zakres MVP pozostaje czytelny dzięki porównaniu „miało być / jest” w dokumencie `MVP.md`, a infrastruktura produkcyjna — dzięki rozdzieleniu ról web, API i bazy oraz świadomym decyzjom o Oracle i Cloudflare Tunnel. Ryzyka utrzymuje się w rejestrze jakościo-

Tabela 6.4: Analiza ryzyka RacePortal — wybrane zagrożenia i sposoby ich ograniczenia.

Ryzyko	**Skutek / objaw**	**Sposób ograniczenia ryzyka**
Niespójność Specyfikacji Formularzy względem implementacji	Luki odbiorcze (m.in. wymagalność danych osobowych)	Rejestr luk w `docs/spec-conformity.md`; rozróżnienie „działa funkcjonalnie” / „pełna zgodność ze Spec”
Awaria lub limity zewnętrznych serwisów map i routingu	Brak mapy lub trasy	Leaflet/OSM bez klucza; OSRM z komunikatami błędu; możliwa wymiana dostawcy
Dryf kontraktu web ↔ mobile	Rozjazdy typów i adresów API	Wspólny pakiet `packages/api-types`
Rozrost zakresu ponad MVP	Opóźnienie odbioru MVP	Świadome odłożenie PSP, push i store; elementy ponad MVP nazywane osobno
Ograniczenia zasobowe i czasowe zespołu	Niewykonanie planu przy budżecie zerowym	Oracle Always Free; jeden origin przez Tunnel; termin karty KP 31.01.2026 jako ograniczenie planistyczne (nie termin domknięcia systemu)
Niekompletność bazy wydarzeń na starcie	Słaba wartość katalogu bez społeczności	Dane demo na start, katalog torów PL, podstawowe ścieżki ról; kompletność rynkowa poza czystym zakresem kodu
Brak formalnego artefaktu 10k/50 RPS	Niespełnienie pierwotnego kryterium odbiorczego KP/DZW	Paginacja, indeksy, cache; pomiary k6 na VM (`docs/testy/wyniki/k6-vm-rps.md` [8]; rozdz. 14)
Zależność od dostarczalności poczty	Rejestracja / statusy „nie działają” dla użytkownika	SMTP Mailbux zamiast Mailpita; diagnostyka SMTP w health (warstwa admin)
Utrata spójności kodu / regresje integracji	Błędy na styku warstw	Code review przed `main`; testy automatyczne API/web/mobile

wym (Spec, wydajność, zależności zewnętrzne), ze sposobami ograniczenia ryzyka osadzonymi w architekturze i dokumentacji porównawczej.

Rozdział 7

Analiza wymagań

7.1 Źródła i sposób pozyskania wymagań

Wymagania dla platformy RacePortal zostały zebrane i sformułowane w oparciu o wielotorowy proces analityczny, ukierunkowany na rozwiązanie realnych problemów rynku motorsportowego. Głównymi źródłami pozyskania wymagań były:

- **Dokument Założeń Wstępnych (DZW) oraz Specyfikacja Wymagań Systemowych (SWS)**, które zdefiniowały biznesowy cel platformy i zakres funkcjonalności dla wersji MVP.
- **Obserwacja polskiego środowiska wydarzeń motorsportowych**, charakteryzującego się dotychczas znacznym rozproszeniem informacji, brakiem spójnych standardów zapisów oraz chaosem komunikacyjnym w mediach społecznościowych.
- **Bezpośrednie rozmowy z kierowcami oraz organizatorami imprez**, które pozwoliły zidentyfikować kluczowe problemy operacyjne, takie jak uciążliwa i powtarzalna weryfikacja techniczna pojazdów czy trudności w efektywnym zarządzaniu listą startową.
- **Osobiste doświadczenia od strony aktywnego kierowcy**, zdobyte przez członka zespołu, Michała Gutowskiego, które dostarczyły bezcennej, praktycznej wiedzy na temat bolączek uczestników i oczekiwań wobec nowoczesnych systemów rejestracji na zawody.

7.2 Wymagania ogólne i dziedzinowe

KARTA WYMAGANIA			
Identyfikator:	W01	Priorytet:	Wysoki
Nazwa:	Zintegrowany ekosystem wydarzeń motorsportowych		
Opis:	System musi agregować dane o imprezach w jednym miejscu, udostępniając je użytkownikom w sposób przejrzysty poprzez klasyczną listę, czytelny kalendarz oraz interaktywną mapę.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.1: Zintegrowany ekosystem wydarzeń motorsportowych

KARTA WYMAGANIA			
Identyfikator:	W02	Priorytet:	Wysoki
Nazwa:	Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów		
Opis:	Platforma ma automatyzować proces zapisów, weryfikując systemowo czy pojazd zawodnika wybrany z jego wirtualnego garażu spełnia regulaminowe wymogi danej kategorii wydarzenia. Odciąża to organizatorów i niweluje błędy ludzkie.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	F04, F05		

Tabela 7.2: Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów

KARTA WYMAGANIA			
Identyfikator:	W03	Priorytet:	Wysoki
Nazwa:	Przejrzysty podział ról w systemie		
Opis:	System opiera się na wyraźnym podziale uprawnień na: użytkowników (szukających startów), organizatorów (zarządzających swoimi imprezami) oraz administratorów (dbających o jakość, wsparcie i bezpieczeństwo platformy).		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	SEC02		

Tabela 7.3: Przejrzysty podział ról w systemie

7.3 Wymagania funkcjonalne

KARTA WYMAGANIA			
Identyfikator:	F01	Priorytet:	Wysoki
Nazwa:	Publiczny katalog i inteligentne filtrowanie		
Opis:	Każdy użytkownik platformy ma dostęp do bazy wydarzeń z intuicyjną możliwością filtrowania ofert po kategorii zawodów, dacie oraz interesującym go województwie.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.4: Publiczny katalog i inteligentne filtrowanie

KARTA WYMAGANIA			
Identyfikator:	F02	Priorytet:	Wysoki
Nazwa:	Szczegóły wydarzenia i wyznaczanie trasy		
Opis:	Platforma udostępnia kompleksowe informacje o imprezie, w tym zintegrowaną funkcję mapy ułatwiającą zawodnikom wyznaczenie najszybszej trasy dojazdu na docelowy obiekt sportowy.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	W01		

Tabela 7.5: Szczegóły wydarzenia i wyznaczanie trasy

KARTA WYMAGANIA			
Identyfikator:	F03	Priorytet:	Wysoki
Nazwa:	Zarządzanie profilem i kontem		
Opis:	Zapewnienie prostego procesu zakładania konta, logowania oraz edycji profilu, umożliwiającego kierowcom sprawne wprowadzanie niezbędnych danych kontaktowych i licencyjnych.		
Udziałowiec:	Kierowca, Organizator		
Wymagania powiązane:	Brak		

Tabela 7.6: Zarządzanie profilem i kontem

KARTA WYMAGANIA			
Identyfikator:	F04	Priorytet:	Wysoki
Nazwa:	Wirtualny Garaż Kierowcy		
Opis:	Cyfrowa ewidencja pojazdów przypisana do profilu. Pozwala na jednorazowe wprowadzenie danych technicznych auta, co drastycznie przyspiesza wielokrotne zapisy na różne zawody w przyszłości.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02		

Tabela 7.7: Wirtualny Garaż Kierowcy

KARTA WYMAGANIA			
Identyfikator:	F05	Priorytet:	Wysoki
Nazwa:	Szybkie aplikowanie na wydarzenia		
Opis:	Kierowca może zgłosić chęć udziału w wybranych zawodach poprzez wskazanie odpowiedniego pojazdu z garażu. Poprawne zgłoszenie natychmiast trafia do organizatora ze statusem oczekującym na akceptację.		
Udziałowiec:	Kierowca		
Wymagania powiązane:	W02, F04		

Tabela 7.8: Szybkie aplikowanie na wydarzenia

KARTA WYMAGANIA			
Identyfikator:	F06	Priorytet:	Średni
Nazwa:	Skuteczny system powiadomień		
Opis:	Wdrożenie powiadomień systemowych wewnątrz aplikacji oraz wiadomości e-mail, które na bieżąco informują użytkowników o istotnych zmianach, takich jak akceptacja lub odrzucenie wniosku o start.		
Udziałowiec:	Zalogowany Użytkownik		
Wymagania powiązane:	Brak		

Tabela 7.9: Skuteczny system powiadomień

KARTA WYMAGANIA			
Identyfikator:	F07	Priorytet:	Wysoki
Nazwa:	Panel Zarządzania dla Organizatora		
Opis:	Dedykowane narzędzie biznesowe dla twórców imprez, pozwalające na łatwą publikację nowych wydarzeń oraz masowe zarządzanie listą zgłoszonych kierowców i ich opłatami.		
Udziałowiec:	Organizator		
Wymagania powiązane:	W03		

Tabela 7.10: Panel Zarządzania dla Organizatora

KARTA WYMAGANIA			
Identyfikator:	F08	Priorytet:	Wysoki
Nazwa:	Panel Administratora i wsparcie użytkowników		
Opis:	Centralne miejsce dowodzenia pozwalające na weryfikację publikowanych treści, nadawanie uprawnień nowym organizatorom oraz sprawną obsługę zgłoszeń i problemów napotkanych przez użytkowników.		
Udziałowiec:	Administrator		
Wymagania powiązane:	W03		

Tabela 7.11: Panel Administratora i wsparcie użytkowników

KARTA WYMAGANIA			
Identyfikator:	F09	Priorytet:	Średni
Nazwa:	Mobilna aplikacja wizytówkowa		
Opis:	Dodatkowy punkt styku dla użytkowników smartfonów, ułatwiający szybkie przeglądanie katalogu imprez w podróży z opcją bezproblemowego przekierowania do głównej platformy webowej w celu sfinalizowania zapisu.		
Udziałowiec:	Gość, Kierowca		
Wymagania powiązane:	Brak		

Tabela 7.12: Mobilna aplikacja wizytówkowa

7.4 Wymagania niefunkcjonalne

KARTA WYMAGANIA			
Identyfikator:	NF01	Priorytet:	Wysoki
Nazwa:	Wydajność i responsywność platformy		
Opis:	System musi działać płynnie, gwarantując krótki czas ładowania katalogu i bezproblemową obsługę nawet przy dużym zainteresowaniu wydarzeniami i wzmożonym ruchem sieciowym.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.13: Wydajność i responsywność platformy

KARTA WYMAGANIA			
Identyfikator:	NF02	Priorytet:	Wysoki
Nazwa:	Dostępność wieloplatformowa (RWD)		
Opis:	Interfejs aplikacji musi automatycznie dostosowywać się do rozmiaru ekranu użytkownika, zapewniając pełen komfort korzystania na smartfonach, tabletach oraz komputerach stacjonarnych.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.14: Dostępność wieloplatformowa (RWD)

KARTA WYMAGANIA			
Identyfikator:	NF03	Priorytet:	Średni
Nazwa:	Niezawodność i łatwość wdrażania		
Opis:	Platforma powinna zapewniać wysoką bezawaryjność oraz pozwalać na łatwe instalowanie nowych funkcjonalności bez długotrwałych przerw w dostępności usług dla kierowców.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.15: Niezawodność i łatwość wdrażania

7.5 Wymagania bezpieczeństwa

KARTA WYMAGANIA			
Identyfikator:	SEC01	Priorytet:	Wysoki
Nazwa:	Bezpieczeństwo danych osobowych i kont		
Opis:	Zaawansowana ochrona kont użytkowników, bezpieczne uwierzytelnianie oraz rygorystyczne szyfrowanie haseł, gwarantujące pełną zgodność z wymogami ochrony prywatności (RODO).		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.16: Bezpieczeństwo danych osobowych i kont

KARTA WYMAGANIA			
Identyfikator:	SEC02	Priorytet:	Wysoki
Nazwa:	Ścisła kontrola dostępu do zasobów		
Opis:	Gwarancja systemowa, że kierowca widzi i zarządza wyłącznie własnymi pojazdami oraz zgłoszeniami, a organizator posiada uprawnienia do wglądu i edycji tylko swoich wydarzeń.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	W03		

Tabela 7.17: Ścisła kontrola dostępu do zasobów

KARTA WYMAGANIA			
Identyfikator:	SEC03	Priorytet:	Średni
Nazwa:	Odporność na nadużycia platformy		
Opis:	Implementacja mechanizmów chroniących platformę przed zautomatyzowanym spamem, atakami masowymi oraz próbami podszywania się pod oficjalną obsługę platformy w celu wyłudzenia informacji.		
Udziałowiec:	Administrator		
Wymagania powiązane:	Brak		

Tabela 7.18: Odporność na nadużycia platformy

7.6 Wymagania interfejsu użytkownika

KARTA WYMAGANIA			
Identyfikator:	UI01	Priorytet:	Wysoki
Nazwa:	Nowoczesny design branżowy		
Opis:	Atrakcyjny wizualnie, spójny interfejs oparty na ciemnym motywie z mocnymi, kontrastującymi akcentami, który naturalnie wpisuje się w dynamikę i estetykę branży motorsportowej.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.19: Nowoczesny design branżowy

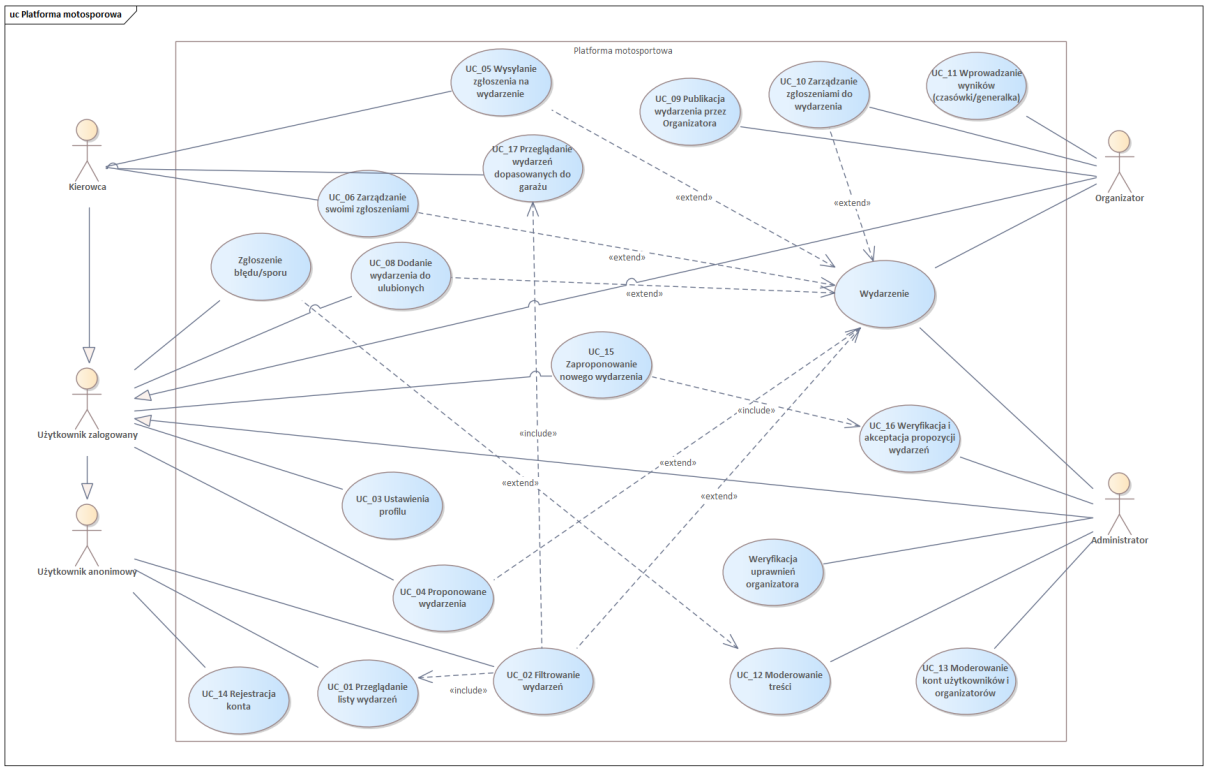
KARTA WYMAGANIA			
Identyfikator:	UI02	Priorytet:	Wysoki
Nazwa:	Intuicyjność i czytelność procesów		
Opis:	Kluczowy nacisk na jasne komunikaty dla użytkownika, widoczne wskaźniki ładowania oraz w pełni zrozumiałe formularze zapobiegające frustracji i pomyłkom podczas rejestracji na ważne zawody.		
Udziałowiec:	Wszyscy		
Wymagania powiązane:	Brak		

Tabela 7.20: Intuicyjność i czytelność procesów

7.7 Diagramy UML

7.7.1 Diagram przypadków użycia

Wczesny diagram przypadków użycia (materiały projektowe) przedstawia rys. 7.1.



Rysunek 7.1: Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna).

7.7.2 Diagram sekwencji: rejestracja i logowanie

7.7.3 Diagram sekwencji: zapis na wydarzenie

7.7.4 Diagram sekwencji: zarządzanie garażem

Uwaga: diagram klas / model danych przeniesiono do projektu systemu — podrozdział 8.4.1.

Rozdział 8

Projekt systemu

Rozdział przedstawia projekt architektury RacePortal: założenia warstwowe, podział logiczny, wdrożenie fizyczne, model danych, kontrakt REST oraz projekt aplikacji mobilnej i mechanizmów bezpieczeństwa. Projekt warstwy webowej (w tym prototyp Figma) omówiono osobno w podrozdziale 8.6. Celem jest pokazanie, *jak* system został zaprojektowany przed opisem realizacji i szczegółów implementacji.

8.1 Założenia architektoniczne

Przyjęto klasyczną architekturę trójwarstwową z jednym kontraktem API dla klientów:

- **warstwa prezentacji** — aplikacja webowa (React/Vite, serwowana statycznie) oraz aplikacja mobilna Expo (wizytówka katalogu);
- **warstwa aplikacji** — API REST (Spring Boot) z logiką domenową, walidacją i kontrolą dostępu;
- **warstwa danych** — relacyjna baza MySQL (JPA / Spring Data).

Kluczowe założenia projektowe:

- jeden origin produkcyjny (`https://raceportal.pl`): nginx serwuje SPA i przekazuje `/api` do backendu — bez osobnego hostingu frontu z osobnym URL API w buildzie;
- wspólny kontrakt JSON oraz pakiet typów `packages/api-types` ograniczający dryf web $\leftrightarrow$ mobile;
- konteneryzacja Docker Compose jako źródło prawdy uruchomieniowej (lokalnie i na VM);
- świadomy podział odpowiedzialności: pełny produkt na webie (w tym PWA), mobilka jako kanał discovery z CTA zapisu na web;

- bezpieczeństwo bazowe wbudowane w projekt API (JWT, RBAC, walidacja wejścia), a TLS na brzegu sieci (Cloudflare).

8.2 Architektura logiczna systemu

Logicznie system dzieli się na moduły domenowe współpracujące przez warstwę serwisów backendu (bez bezpośredniego dostępu klientów do bazy):

- **użytkownicy i auth** — rejestracja, weryfikacja e-mail, logowanie hasłem / Google, odświeżanie sesji, profil;
- **wydarzenia** — katalog, detal, statusy cyklu życia, archiwizacja, licznik wyświetleń;
- **zgłoszenia** — flow statusów, dowód płatności, przypisanie klasy;
- **garaż** — pojazdy użytkownika i dopasowanie do wymagań wydarzenia (`carMatch`);
- **organizator** — workspace wydarzenia: klasy, wyniki, komunikacja, eksporty;
- **administracja** — moderacja wydarzeń i ról oraz backoffice na osobnym hoście;
- **mapy / trasa** — markery katalogu i wyznaczanie dojazdu (OSRM);
- **powiadomienia i poczta** — powiadomienia in-app oraz maile transakcyjne (SMTP);
- **pliki** — upload obrazów i PDF (m.in. dowody wpłaty, regulaminy) z odczytem przez API.

Podział odpowiada granicom uprawnień RBAC: gość korzysta z odczytu publicznego, kierowca ze zgłoszeń i garażu, organizator z własnych wydarzeń, administrator z moderacji całej platformy.

8.3 Architektura fizyczna i wdrożeniowa

Środowisko uruchomieniowe oparto o Docker Compose. Lokalnie (i analogicznie na VM) pracują kontenery:

- `web` — nginx ze zbudowanym SPA oraz reverse proxy `/api`;
- `api` — Spring Boot (JVM);
- `mysql` — baza trwała (volume);
- `backup` — periodyczne kopie zapasowe bazy.

Aplikacja mobilna Expo działa poza Compose (Expo Go / emulator). Poczta wychodząca korzysta z zewnętrznego SMTP Mailbux (poza Compose); dane dostępowe SMTP są trzymane w `.env`, poza repozytorium.

Wdrożenie produkcyjne: maszyna Oracle Always Free (ARM) z tym samym stackiem Compose. Publiczny dostęp zapewnia **Cloudflare Tunnel** (`cloudflared`) do nginx na porcie `:8081`. Apex `https://raceportal.pl` oraz host backoffice są w koncie Cloudflare — TLS/HSTS i DNS po stronie Cloudflare, bez hostowania frontu na Cloudflare Pages. Dzięki temu deploy sprowadza się do aktualizacji kodu na VM i przebudowy kontenerów, a klienci wołają API względną ścieżką `/api`.

8.4 Projekt bazy danych

Model danych zaprojektowano wokół wydarzenia motorsportowego i procesu zapisu. Główne encje implementacyjne obejmują m.in.:

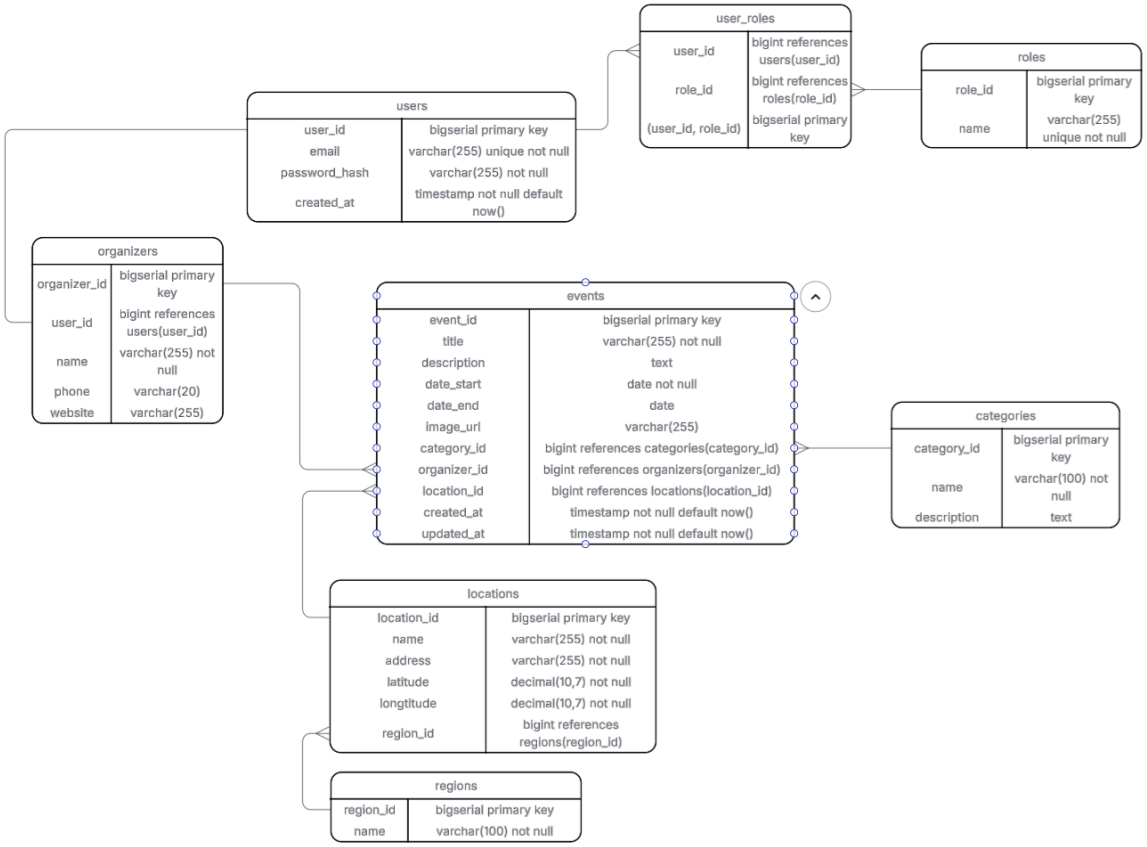
- użytkownika (rola jako enum: `USER` / `ORGANIZER` / `ADMIN`);
- pojazd w garażu;
- wydarzenie (lokalizacja i współrzędne, status cyklu życia, opłaty, regulaminy, flaga publikacji wyników, licznik wyświetleń);
- zgłoszenie (status, dowód płatności, klasa zawodów);
- tor z katalogu PL (seed przy starcie API; odczyt listy torów przez API);
- klasy zawodów, wyniki i reguły punktacji powiązane z wydarzeniem;
- wniosek o rolę organizatora, powiadomienia, refresh tokeny oraz zgłoszenia błędów.

Wydarzenie przechodzi przez kolejne stany: od szkicu i moderacji, przez publikację w katalogu, aż do archiwum albo anulowania. Zgłoszenie kierowcy ma osobne stany: najpierw czeka na decyzję organizatora (`PENDING`), potem może zostać wstępnie przyjęte (`ACCEPTED`) albo od razu potwierdzone (`CONFIRMED`); anulowanie oznacza `CANCELED`. Przy wydarzeniach płatnych po `ACCEPTED` kierowca dokłada dowód przelewu (model „zapłać później”), a organizator dopiero wtedy potwierdza miejsce. Listy torów w formularzach web i mobile korzystają ze wspólnego zestawu nazw, żeby obie aplikacje pokazywały te same lokalizacje.

8.4.1 Diagram klas

Rysunek 8.1 przedstawia wczesny model bazy danych (ERD) stanowiący punkt wyjścia projektu klas / tabel domenowych. W implementacji Spring Data JPA model został uproszczony

i dostosowany do MVP (m.in. role jako enum użytkownika, lokalizacja osadzona w wydarzeniu). Kolejne encje operacyjne (klasy, wyniki, uploady jako URL-e w polach domenowych) rozbudowano iteracyjnie bez zmiany ogólnego wzorca: jedna baza MySQL, relacje JPA, statusy jako enumy.



Rysunek 8.1: Diagram klas / model danych (ERD) — wersja koncepcyjna projektu.

8.5 Projekt interfejsu REST API

Interfejs API zaprojektowano w stylu REST nad protokołem HTTP, z wymianą danych w formacie JSON. Prefiks zasobów to `/api/...`. Kontrakt odpowiedzi jest wspólny dla klienta webowego i mobilnego; typy współdzielone utrzymywane są w pakiecie `packages/api-types`, co ogranicza rozjazd interpretacji pól pomiędzy warstwami.

Zasoby podzielono według odpowiedzialności domenowych. **Uwierzytelnianie i profil** (`/api/auth`) obejmują rejestrację, weryfikację e-mail, logowanie, odświeżanie i unieważnianie sesji, opcjonalne logowanie Google oraz operacje na profilu (`/me`). **Katalog wydarzeń** (`/api/events`) udostępnia listę z paginacją i filtrami, markery mapy, metadane kategorii, szczegóły wydarzenia oraz operacje tworzenia, edycji, anulowania i usuwania dla uprawnionych ról. Uzupełniają je zasoby klas startowych, listy startowej, wyników oraz komunikacji organizatora do uczestni-

ków konkretnego wydarzenia.

Zgłoszenia (`/api/registrations`) umożliwiają utworzenie zgłoszenia, podgląd własnych zapisów, listę zgłoszeń wydarzenia, zmianę statusu, anulowanie, dołączenie dowodu płatności oraz przypisanie klasy. **Garaż** (`/api/garage`) obsługuje CRUD pojazdów użytkownika. **Organizator** (`/api/organizer`) obejmuje wniosek o rolę oraz listę zarządzanych wydarzeń, a **administrator** (`/api/admin`) — statystyki, użytkowników, moderację wydarzeń i rozstrzyganie wniosków organizatorskich.

Pozostałe zasoby wspierające to: katalog torów (`/api/tracks`), wyznaczanie trasy (`/api/maps/route`), upload plików (`/api/uploads`), powiadomienia in-app (`/api/notifications`), publiczne statystyki (`/api/stats`), zgłoszenia błędów (`/api/bug-reports`) oraz monitoring (`/api/health`).

Z punktu widzenia dostępu projekt rozróżnia zasoby publiczne (m.in. odczyt wydarzeń i torów, health, trasa, rejestracja/logowanie) oraz zasoby wymagające uwierzytelnienia i odpowiedniej roli. Błędy zwracane są w ujednoliconym formacie JSON (walidacja pól, kody HTTP 4xx/5xx), co upraszcza obsługę po stronie klientów. Szczegóły implementacji endpointów i przepływów opisano w rozdziale 10.

8.6 Projekt warstwy webowej

Warstwa webowa RacePortal [1] to pełny klient transakcyjny systemu: obsługuje katalog wydarzeń, konta użytkowników, wirtualny garaż, zgłoszenia udziału oraz panele organizatora i administratora. Stack aplikacji to **Vite 6** [9] z **React 18** i **TypeScript**, stylowanie **Tailwind CSS v4** oraz zestaw komponentów zbliżonych do **shadcn/ui** (Radix UI). Routing realizuje **React Router v7** (`createBrowserRouter`); komunikacja z backendem — moduł komunikacji z API w `web/src/app/lib/api.ts`, z bazą URL z `VITE_API_URL` albo ścieżkami względnymi `/api/...` (proxy Vite w dev, proxy nginx w Dockerze). Sesja opiera się o **httpOnly cookies** (`rp_at` / `rp_rt`) z `credentials: "include"`; starszy JWT w `localStorage` jest usuwany przy starcie. Szczegóły implementacji frontendu są rozwinięte w rozdziale 11; poniżej opisano projekt warstwy i kluczowe decyzje.

Architektura frontendu

Korzeń aplikacji (`App.tsx`) owija drzewo tras w `AuthProvider`, wczytuje ustawienia UI (akcent / język) oraz renderuje `RouterProvider`. Strona główna ładuje się od razu (LCP); pozostałe strony — przez `React.lazy`, z obsługą błędu po aktualizacji (jednorazowe odświeżenie strony). Wspólny układ zapewnia `RootLayout` z `<Outlet />`. Trasy chronione opakowano w `AuthGate`: podczas wczytywania sesji (`GET /api/auth/me`) wyświetlany jest spinner; brak sesji przekierowuje na `/login`; opcjonalna biała lista ról (`ADMIN`, `ORGANIZER`) chroni panele. W Dockerze nginx stosuje `try_files $uri /index.html`, aby bezpośrednie odnośniki SPA

nie kończyły się kodem 404.

Szkic kluczowych tras (fragment `routes.tsx`):

```
export const router = createBrowserRouter([
  {
    path: "/",
    Component: RootLayout,
    children: [
      { index: true, Component: HomePage },
      { path: "login", Component: LoginPage },
      { path: "wydarzenia", Component: EventsPage },
      { path: "wydarzenia/:id", Component: EventDetailPage },
      { path: "dashboard", Component: ProtectedDashboard },
      { path: "garaz", Component: ProtectedGarage },
      { path: "organizer", Component: ProtectedOrganizer },
      { path: "admin", Component: ProtectedAdmin },
      // ...
    ],
  },
]);
```

Fragment kodu 8.1: Fragment drzewa tras React Router (katalog, auth, panele).

Komunikacja z API i stan sesji

Moduł `api` zbiera metody `get/post/patch/delete`, mapuje błędy na `ApiError` oraz przy kodzie 401 wykonuje jedną próbę odświeżenia sesji (`POST /api/auth/refresh`) przed wylogowaniem. Kontekst `AuthContext` utrzymuje `user`, flagę `isLoading` oraz funkcje konta (logowanie, rejestracja z weryfikacją e-mail, OAuth Google, profil, zmiana hasła, zamknięcie konta). Wczytywanie sesji jest odroczone do idle (`deferUntilIdle`), a przy powrocie karty przeglądarki odświeżany jest token — bez przechowywania JWT w JavaScript.

```
useEffect(() => {
  clearLegacyToken();
  return deferUntilIdle(() => {
    void (async () => {
      try {
        const me = await api.get<User>("/api/auth/me");
        persistAuth(me);
      } catch {
        persistAuth(null);
      } finally {
        setIsLoading(false);
      }
    })();
  }, 2000);
}, []);
```

Fragment kodu 8.2: Wczytywanie sesji w `AuthContext` (httpOnly cookies).

Wspólny kontrakt typów z backendem wspiera pakiet `packages/api-types` (m.in. presety

torów `TRACK_PRESETS`). Mapa wydarzeń korzysta z **Leaflet** + OpenStreetMap; wyznaczanie trasy — z publicznego OSRM (Google Maps pozostaje opcją konfiguracyjną poza domyślną ścieżką MVP).

Komponenty i użyteczność

Struktura katalogów oddziela strony (`pages/`), kontekst auth, moduł komunikacji z API oraz bibliotekę UI (`components/ui/` — przyciski, dialogi, formularze, karty itd.). Tokeny marki (`--race-accent`, `--race-fg`, `--race-muted`) spójnie sterują motywem. Typografia display to **Oxanium** (zastąpiono Orbitron ze względu na brak *latin-ext* dla polskich znaków), treść — Montserrat. Interfejs jest **mobile-first**: na telefonie działa dolna nawigacja oraz PWA (manifest + service worker, instalacja skrótu na ekranie początkowym). Stany puste, błędy tras i toasty (`sonner`) domykają feedback użytkownika. Panele ról nie dublują katalogu publicznego — organizator zarządza własnymi wydarzeniami, administrator moderuje publikacje i wnioski o rolę.

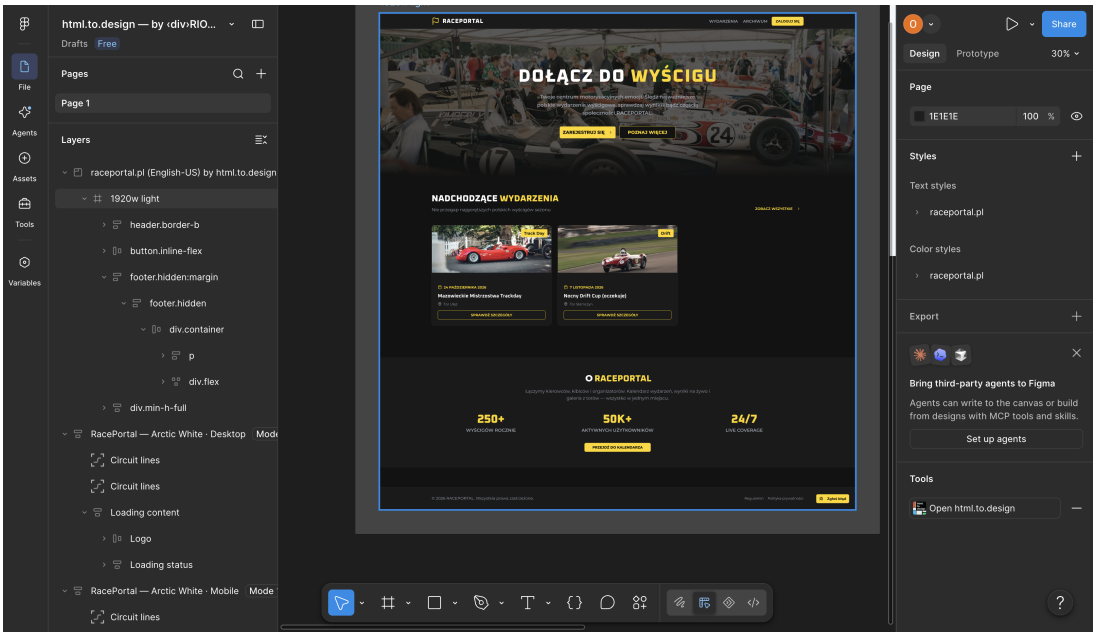
Użyteczność projektowano wokół krótkiej ścieżki: gość przegląda katalog → rejestracja / logowanie → garaż → zapis na wydarzenie; organizator publikuje imprezę po akceptacji administratora. CTA na stronie głównej i w kartach wydarzeń prowadzą do konkretnych tras SPA, a nie do martwych placeholderów prototypu.

8.6.1 Prototyp UI w Figma Make

Wyjściowym projektem interfejsu był plik Figma Make **Advanced UI/UX for RACEPORTAL** (ciemny motyw motorsportowy, akcent marki w kolorze złotym / żółtym, typografia display, komponenty zbliżone do shadcn/ui). Prototyp obejmował m.in. stronę główną z hero i sekcją nadchodzących wydarzeń, katalog, logowanie/rejestrację, dashboard oraz szkice paneli organizatora i administratora. Warstwy w Figmie nosiły nazwy zbliżone do klas narzędziowych CSS (`header.border-b`, `button.inline-flex`), co ułatwiło przeniesienie układu do React/Tailwind.

Rysunek 8.2 przedstawia zrzut ekranu prototypu strony głównej (stan roboczy z Figmy, dokumentacja zespołu z 23.10.2026): hero z hasłem „Dołącz do wyścigu”, karty wydarzeń oraz blok informacyjny z wezwaniem do kalendarza.

Kod frontendu został przeniesiony z Figma Make do lokalnego monorepo (`web/`), a następnie podpięty pod realne API Spring Boot zamiast mocków i `localStorage`. Wczesna wizja produktu i ról — jeszcze przed pełną implementacją — jest też widoczna na diagramach koncepcyjnych (rys. 7.1, 4.1). W toku realizacji UI dopracowano pod kątem responsywności, stanów pustych/błędów, PWA oraz spójności z tokenami marki (`-race-accent`); font Orbitron zastąpiono Oxaniumem ze względu na polskie znaki diakrytyczne. Atrybucje komponentów shadcn/ui oraz zdjęć Unsplash udokumentowano w `docs/ATTRIBUTIONS.md`.



Rysunek 8.2: Prototyp UI RacePortal w Figma Make — strona główna (ciemny motyw motorsportowy, akcent marki).

8.7 Projekt aplikacji mobilnej

Aplikację mobilną zaprojektowano świadomie jako **wizytówkę katalogu**, a nie pełny klon weba. Założenia:

- stack Expo (React Native) z nawigacją typu stack wokół listy i detalu wydarzeń;
- publiczny klient HTTP do tych samych endpointów katalogu co web;
- konfiguracja adresów API i weba przez zmienne środowiskowe Expo (`EXPO_PUBLIC_API_URL`, `EXPO_PUBLIC_WEB_URL`);
- konwersja zapisu: przycisk CTA otwiera detal wydarzenia w przeglądarce / PWA (`Linking` → ścieżka wydarzenia na webie).

Poza zakresem projektu mobilnego pozostają: logowanie, garaż, panele organizatora/admina oraz publikacja w sklepach. Aplikacja mobilna służy więc głównie do przeglądania katalogu; sam zapis na wydarzenie użytkownik dokańcza w wersji webowej.

8.8 Projekt mechanizmów bezpieczeństwa

Bezpieczeństwo zaprojektowano warstwowo, z naciskiem na kontrolę dostępu API:

- uwierzytelnianie JWT — sesja webowa w ciasteczkach httpOnly (`rp_at`/`rp_rt`); API akceptuje też nagłówek Bearer (np. testy) oraz opcjonalne OAuth Google;

- autoryzacja RBAC na endpointach i w serwisach (rozróżnienie właściciela wydarzenia vs administrator);
- hasła przechowywane hashowane (BCrypt); osobne refresh tokeny;
- walidacja DTO po stronie API oraz odrzucanie niebezpiecznych URL-i tam, gdzie przyjmowane są odnośniki;
- CORS ograniczony do skonfigurowanych originów; na brzegu — TLS i nagłówki bezpieczeństwa nginx (m.in. CSP);
- rozdzielenie apexu i backoffice administracyjnego na osobnym hoście, z twardym wymaganiem roli `ADMIN`.

Szczegółową analizę zagrożeń, klasyfikację danych i ograniczenia przyjętych zabezpieczeń rozwinięto w rozdziale 13.

Rozdział 9

Realizacja

Rozdział opisuje, jak projekt z rozdziału 8 przełożono na działający system: organizację prac implementacyjnych, podział na komponenty, sposób ich integracji oraz uruchamianie środowiska (lokalnie i produkcyjnie). Szczegóły kodu backendu, frontendu i mobile rozwijają kolejne rozdziały implementacyjne.

9.1 Organizacja implementacji

Prace prowadzono iteracyjnie we wspólnym monorepo (web, API, mobile, pakiet typów, dokumentacja). Każda istotniejsza zmiana była weryfikowana uruchomieniem Compose oraz — w miarę możliwości — testami automatycznymi. Środowisko Docker Compose traktowano jako źródło prawdy uruchomieniowej: to, co działa w kontenerach lokalnie, jest tym samym stackiem co na VM produkcyjnej.

Organizacja pracy zakładała:

- dostarczanie kolejnych ścieżek użytkownika (katalog → konto → garaż/zapisy → panele ról), a nie budowę wszystkich warstw „na zapas”;
- bieżące dokumentowanie decyzji i luk (m.in. rejestr zgodności ze Specyfikacją Formularzy);
- świadome odkładanie elementów poza MVP (m.in. natywne PSP, pełna parity mobile, sklepy aplikacji);
- utrzymanie seedu danych demo, aby prezentacja i testy E2E miały powtarzalny punkt startu.

9.2 Podział systemu na komponenty

System zrealizowano jako trzy główne komponenty kliencko-serwerowe plus warstwę danych. Backend i frontend webowy omówiono skrótowo poniżej (szczegóły — rozdz. 10 i 11); aplika-

cję mobilną — skrótowo poniżej, a szczegółowo w rozdz. 12.

9.2.1 Backend

Komponent backendowy stanowi warstwę serwerową RacePortal i odpowiada za udostępnienie API REST, trwałe przechowywanie danych oraz egzekwowanie reguł biznesowych i uprawnień. Został zrealizowany w technologii Spring Boot (wersja 3.3.5) na platformie Java 21, z relacyjną bazą MySQL oraz mapowaniem obiektowo-relacyjnym Spring Data JPA. Budowa i uruchamianie odbywają się w kontenerze (obraz Maven do kompilacji, obraz JRE do runtime), co zapewnia powtarzalność środowiska lokalnego i produkcyjnego.

Struktura kodu w pakiecie `pl.raceportal` odzwierciedla klasyczny podział warstwowy. Warstwa kontrolerów eksponuje zasoby HTTP (m.in. uwierzytelnianie, wydarzenia, zgłoszenia, garaż, tory, mapy, uploady, panele organizatora i administratora oraz health-check). Warstwa serwisów zawiera logikę dziedzinową — m.in. rejestrację i logowanie, zarządzanie wydarzeniami, przepływ statusów zgłoszeń, dopasowanie pojazdu do wymagań wydarzenia, powiadomienia e-mail, moderację oraz archiwizację. Warstwa repozytoriów zapewnia dostęp do encji domenowych (użytkownik, pojazd, wydarzenie, zgłoszenie, tor i encje powiązane). Obiekty DTO oraz walidacja Bean Validation oddzielają kontrakt API od modelu persystencji, a globalna obsługa wyjątków ujednolica odpowiedzi błędów.

Bezpieczeństwo dostępu oparto o Spring Security oraz tokeny JWT: część zasobów katalogu pozostaje publiczna (przeglądanie wydarzeń, torów, trasy), natomiast operacje konta, garażu, zgłoszeń i moderacji wymagają uwierzytelnienia oraz odpowiednich ról. Komponent backendowy realizuje również zadania wspierające demonstrację i utrzymanie systemu: inicjalizację danych startowych (`DataInitializer`), katalog torów, cache wybranych list, a także cykliczną archiwizację minionych wydarzeń (`ArchiveService`, uruchamiana przy starcie aplikacji oraz codziennie według harmonogramu).

W podziale odpowiedzialności backend jest jedynym źródłem prawdy dla stanu domeny — klienci webowy i mobilny nie utrzymują własnej bazy wydarzeń ani zgłoszeń, lecz konsumują wspólny kontrakt JSON. Szczegółowy opis encji, endpointów i przepływów implementacyjnych przedstawiono w rozdziale 10.

9.2.2 Frontend webowy

Frontend webowy RacePortal został przygotowany w React z wykorzystaniem TypeScript i narzędzia Vite. Aplikacja składa się z osobnych stron oraz komponentów, a z backendem komunikuje się przez REST API. Użytkownik może przeglądać katalog wydarzeń i sprawdzać szczegóły wybranej imprezy. Po zalogowaniu kierowca ma również dostęp do swojego profilu oraz garażu, w którym może dodawać i edytować pojazdy.

Dostęp do poszczególnych funkcji zależy od roli użytkownika. Organizator korzysta z osobnego panelu do zarządzania wydarzeniami i zgłoszeniami, a administrator posiada panel do moderowania treści i zarządzania uprawnieniami. Aplikacja sprawdza, czy użytkownik jest zalogowany i czy ma odpowiednie uprawnienia do otwarcia danej strony. Dzięki temu jedna aplikacja łączy publiczny katalog wydarzeń z funkcjami dostępnymi po zalogowaniu.

9.2.3 Aplikacja mobilna

Komponent mobilny zrealizowano w Expo jako publiczną wizytówkę katalogu:

- lista i detal wydarzeń na tych samych danych API co web;
- brak logowania, garażu i paneli organizatora/admina;
- CTA „Zapisz się na stronie" przekierowujące do detalu wydarzenia w wersji webowej.

Aplikacja mobilna służy więc głównie do przeglądania katalogu; sam zapis na wydarzenie użytkownik dokańcza w wersji webowej.

9.3 Integracja komponentów

Integracja opiera się na wspólnym kontrakcie REST/JSON. Pakiet typów TypeScript (`packages/api-types`) ogranicza rozjazdy pól między webem a mobile. W środowisku Compose nginx w kontenerze `web` serwuje SPA i przekazuje żądania `/api` do kontenera `api`, dzięki czemu przeglądarka korzysta z jednego originu (bez osobnego URL API w buildzie produkcyjnym).

Przepływ typowy: klient (web lub mobile) woła endpoint API $\rightarrow$ Spring realizuje reguły domenowe i RBAC $\rightarrow$ MySQL utrwala stan $\rightarrow$ opcjonalnie wychodzi mail transakcyjny (SMTP Mailbux). Pliki (obrazy, PDF) trafiają do volume uploadów i są odczytywane przez API.

9.4 Konteneryzacja i uruchamianie środowiska

Uruchomienie lokalne sprowadza się do zbudowania i startu stacku Compose, a następnie sprawdzenia zdrowia API. Mobilkę uruchamia się osobno w katalogu Expo. W skrócie:

- start stacku: `docker compose up -build -d` [10];
- weryfikacja: endpoint health API zwraca stan OK;
- mobile: start Expo w katalogu aplikacji mobilnej;
- produkcja: ten sam Compose na Oracle VM, a ruch publiczny przez Cloudflare Tunnel do nginx na porcie `:8081`.

Dzięki zastosowaniu tego samego stacku kontenerów w środowisku deweloperskim i produkcyjnym wdrożenie nowej wersji polega na aktualizacji kodu na maszynie wirtualnej oraz przebudowie usług Docker Compose. Aplikacja webowa i API są publikowane łącznie, bez niezależnego procesu wdrożenia frontendu.

Rozdział 10

Szczegóły implementacji backendu

Rozdział niniejszy uszczegóławia realizację warstwy serwerowej RacePortal, której ogólny zarys przedstawiono w podrozdziale 9.2.1. Opis oparto o faktyczną strukturę pakietu `pl.raceportal` oraz zachowanie API REST.

10.1 Struktura projektu backendowego

Backend jest osobnym modułem Maven (`raceportal-backend`) w monorepozytorium. Rodzic projektu to `spring-boot-starter-parent` w wersji 3.3.5; język implementacji to Java 21. Do kluczowych zależności należą Spring Web, Spring Security, Spring Data JPA, Bean Validation, Spring Mail, sterownik MySQL, cache Caffeine oraz biblioteka JJWT. Opcjonalne logowanie Google wykorzystuje klienta Google API.

Kod aplikacji zorganizowano w pakietach odpowiadających odpowiedzialnościom:

- `controller` — ekspozycja zasobów HTTP;
- `service` — logika biznesowa i integracje (m.in. poczta, mapy, upload);
- `repository` — dostęp do danych przez Spring Data JPA;
- `domain` — encje i enumy statusów/ról;
- `dto` — kontrakty żądań i odpowiedzi z adnotacjami walidacji;
- `security` — JWT, ciasteczka sesji, limity prób, konfiguracja Spring Security;
- `web` — wyjątki domenowe i globalna obsługa błędów;
- `config`, `i18n` — konfiguracja serializacji oraz teksty wiadomości e-mail.

Klasa startowa `RacePortalApplication` włącza harmonogram zadań (`@EnableScheduling`), wykorzystywany m.in. do archiwizacji wydarzeń.

10.2 Model domenowy i encje

Model persystencji odzwierciedla główne pojęcia katalogu i zapisów. Encja `User` reprezentuje konto: unikalny e-mail i nazwę użytkownika, skrót hasła, rolę (`USER`, `ORGANIZER`, `ADMIN`), dane profilu, flagi weryfikacji e-mail oraz pola związane z opcjonalnym kontem Google. Encja `Car` opisuje pojazd w garażu (marka, model, klasa, napęd, parametry techniczne, opony, wymagane dokumenty/wyposażenie). Encja `Event` przechowuje opis wydarzenia, lokalizację (w tym współrzędne), kategorię, status publikacji, warunki płatności oraz wymagania wobec uczestnika i pojazdu; wydarzenie jest powiązane z organizatorem (`User`).

Encja `Registration` modeluje zgłoszenie udziału: powiązanie użytkownika, wydarzenia i pojazdu, status, komentarze, dowód płatności oraz termin opłaty. Statusy zgłoszenia obejmują m.in. `PENDING`, `ACCEPTED`, `CONFIRMED` i `CANCELED`. Statusy wydarzenia obejmują m.in. `DRAFT`, `PENDING`, `APPROVED`, `REJECTED`, `ARCHIVED` i `CANCELLED`.

Dodatkowo w modelu występują: katalog torów (`Track`), wnioski o rolę organizatora (`OrganizerApplica`
odświeżające tokeny sesji (`RefreshToken`), klasy zawodów powiązane z wydarzeniem, wyniki, reguły punktacji, powiadomienia in-app oraz zgłoszenia błędów. Katalog torów nie jest kluczem obcym wydarzenia — lokalizacja wydarzenia jest przechowywana w polach wydarzenia, a katalog służy jako słownik wspomagający formularze i filtry. Fragment encji `User` (listing 10.1) ilustruje kluczowy model tożsamości i ról.

```
1  @Entity
2  @Table(name = "users")
3  public class User {
4      @Id
5      @GeneratedValue(strategy = GenerationType.UUID)
6      private String id;
7
8      @Column(nullable = false, unique = true, length = 190)
9      private String email;
10
11     @Column(name = "password_hash", nullable = false, length = 100)
12     private String passwordHash;
13
14     @Enumerated(EnumType.STRING)
15     @Column(nullable = false, length = 20)
16     private Role role = Role.USER;
17     // ... profil, weryfikacja e-mail, tokenVersion
18 }
```

Fragment kodu 10.1: Kluczowe pola encji `User`

10.3 Rejestracja użytkowników i logowanie

Uwierzytelnianie zrealizowano w `AuthController` (`/api/auth`) oraz `AuthService`. Rejestracja kierowcy tworzy konto z rolą `USER` w stanie niezweryfikowanym i wysyła wiadomość

aktywacyjną z linkiem `/verify-email/{token}` (czas ważności tokenu ograniczony). Do momentu weryfikacji system nie wydaje sesji użytkownika. Dostępna jest także ścieżka rejestracji organizatora, która równolegle zakłada wniosek o uprawnienia.

Logowanie hasłem weryfikuje poświadczenia i wystawia sesję: krótkotrwały token dostępu JWT oraz rotowany token odświeżający przechowywany w bazie w postaci skrótu. Token dostępu niesie m.in. identyfikator, e-mail, nazwę użytkownika, rolę oraz wersję tokenu (`tokenVersion`), umożliwiając unieważnienie sesji przy wylogowaniu lub zmianie poświadczeń. Odświeżanie sesji odbywa się endpointem `/refresh`; wylogowanie unieważnia refresh i podnosi wersję tokenu. Opcjonalnie wspierane jest logowanie Google (weryfikacja `idToken` przy skonfigurowanym identyfikatorze klienta).

Ochronę przed nadużyciami zapewnia `AttemptRateLimiter` (m.in. limity prób logowania i weryfikacji). Konfiguracja Spring Security utrzymuje sesję bezstanową po stronie serwera i udostępnia publicznie m.in. endpointy rejestracji, logowania, weryfikacji, odświeżania oraz odczyt katalogu wydarzeń (listing 10.2). Wydanie sesji po udanym logowaniu lub weryfikacji e-mail realizuje `issueSession` (listing 10.3).

```
http
  .csrf(csrf -> csrf.disable())
  .sessionManagement(sm ->
      sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers(HttpMethod.POST,
          "/api/auth/login", "/api/auth/register",
          "/api/auth/verify-email", "/api/auth/refresh").permitAll()
      .requestMatchers(HttpMethod.GET, "/api/events/**").permitAll()
      .anyRequest().authenticated())
  .addFilterBefore(jwtAuthFilter,
      UsernamePasswordAuthenticationFilter.class);
```

Fragment kodu 10.2: Fragment `SecurityFilterChain` — sesja bezstanowa i ścieżki publiczne

```
private AuthResponse issueSession(User user) {
    String access = jwtService.generateToken(user);
    String refresh = refreshTokenService.issue(user);
    return new AuthResponse(access, refresh, toUserDto(user));
}
```

Fragment kodu 10.3: Wydanie sesji: JWT access + rotowany refresh

10.4 Zarządzanie profilami użytkowników

Profil zalogowanego użytkownika jest obsługiwany w ramach zasobów `/api/auth` (brak osobnego kontrolera profilu). Endpoint `GET /me` zwraca bieżące dane konta, a `PATCH /me` umożliwia aktualizację nazwy użytkownika, avatara, imienia i nazwiska, telefonu, adresu, odnośnika

do Instagrama, deklaracji prawa jazdy kategorii B i licencji PZM oraz preferencji lokalizacji językowej i motywu interfejsu.

Zmiana hasła (`POST /me/password`) jest dostępna dla kont lokalnych i zablokowana dla kont utworzonych wyłącznie przez Google. Zmiana adresu e-mail przebiega dwuetapowo (żądanie + potwierdzenie), z powiadomieniem na dotychczasowy adres. Użytkownik może zamknąć własne konto po podaniu wymaganej frazy potwierdzającej. Takie podejście łączy zarządzanie tożsamością z kontem w jednym module autentycznym, upraszczając powierzchnię API (listing 10.4).

```
@GetMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> me(
        @AuthenticationPrincipal UserPrincipal currentUser) {
    return ResponseEntity.ok(authService.me(currentUser.getId()));
}

@PatchMapping("/me")
@PreAuthorize("isAuthenticated()")
public ResponseEntity<UserDto> updateMe(
        @AuthenticationPrincipal UserPrincipal currentUser,
        @Valid @RequestBody UpdateMeRequest request) {
    return ResponseEntity.ok(
        authService.updateMe(currentUser.getId(), request));
}
```

Fragment kodu 10.4: Endpointy profilu w `AuthController`

10.5 Obsługa wydarzeń

Zasób `/api/events` (`EventController`, `EventService`) realizuje publiczny katalog oraz zarządzanie wydarzeniami przez uprawnione role. Lista wydarzeń obsługuje paginację oraz filtry (m.in. tekst, kategoria, miasto, województwo, tor, zakres dat, status, płatność, dopasowanie do pojazdu z garażu). Publicznie domyślnie widoczne są wydarzenia zatwierdzone z datą bieżącą lub przyszłą. Osobny endpoint markerów dostarcza dane do widoku mapy.

Tworzenie i edycja wydarzeń wymagają roli organizatora lub administratora. Nowe wydarzenie organizatora trafia do statusu oczekującego na moderację (z powiadomieniem administratorów); administrator może publikować od razu jako zatwierdzone (listing 10.5). Edycja przez organizatora ponownie kieruje treść do moderacji i może uruchomić powiadomienia do osób już zgłoszonych. Anulowanie wydarzenia ustawia odpowiedni status, anuluje otwarte zgłoszenia i informuje uczestników. Uzupełnieniem cyklu życia jest `ArchiveService`, który przy starcie aplikacji oraz codziennie przenosi minione, zatwierdzone wydarzenia do archiwum.

```
@Transactional
@CacheEvict(cacheNames = "events", allEntries = true)
public EventResponse create(EventCreateRequest request,
```

```
                            UserPrincipal currentUser) {
    Event event = new Event();
    // ... mapowanie pol z requestu (nazwa, data, lokalizacja, oplaty)
    event.setStatus(currentUser.getRole() == Role.ADMIN
            ? EventStatus.APPROVED
            : EventStatus.PENDING);
    event.setOrganizer(organizer);
    event = eventRepository.save(event);
    if (event.getStatus() == EventStatus.PENDING) {
        notificationService.notifyAdminsPendingEvent(event);
    }
    return serialize(event);
}
```

Fragment kodu 10.5: Tworzenie wydarzenia — status zależny od roli

10.6 Obsługa zgłoszeń

Zgłoszenia obsługuje `RegistrationController` (`/api/registrations`) przy wymaganiu uwierzytelnienia. Utworzenie zgłoszenia wiąże kierowcę, zatwierdzone wydarzenie przyjmujące zapisy oraz — opcjonalnie — pojazd z garażu; system może przypisać klasę startową według reguł wydarzenia. Początkowy status to `PENDING`.

Zmiany statusów podlegają walidacji przejść zależnej od charakteru wydarzenia. Dla imprez płatnych typowy przebieg obejmuje akceptację (`ACCEPTED`) z wyznaczeniem terminu opłaty, dołączenie dowodu płatności przez kierowcę oraz potwierdzenie (`CONFIRMED`). Dla imprez bezpłatnych możliwe jest przejście z `PENDING` bezpośrednio do `CONFIRMED`. Kierowca może anulować własne zgłoszenie w dopuszczalnych stanach. Operacje listy zgłoszeń wydarzenia oraz zmiany statusów są ograniczone do organizatora-właściciela lub administratora. Kluczowe zmiany generują transakcyjne wiadomości e-mail (utworzenie, status, anulowanie). Maszynę stanów ilustruje listing 10.6.

```
private void validateTransition(RegistrationStatus from,
        RegistrationStatus to, Registration registration, Event event) {
    if (from == RegistrationStatus.CANCELED
            || from == RegistrationStatus.CONFIRMED) {
        throw ApiException.badRequest(
            "Nie mozna zmienic statusu zgloszenia zakonczonego");
    }
    boolean valid = switch (from) {
        case PENDING -> event.isPaid()
            ? (to == RegistrationStatus.ACCEPTED
                || to == RegistrationStatus.CANCELED)
            : (to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED);
        case ACCEPTED -> to == RegistrationStatus.CONFIRMED
                || to == RegistrationStatus.CANCELED;
        default -> false;
    };
    if (!valid) {
```

```
19          throw ApiException.badRequest(
20              "Nieprawidlowe przejscie statusu zgloszenia");
21      }
22      if (to == RegistrationStatus.CONFIRMED
23              && from == RegistrationStatus.ACCEPTED
24              && (registration.getPaymentProofUrl() == null
25                  || registration.getPaymentProofUrl().isBlank())) {
26          throw ApiException.badRequest(
27              "Brak potwierdzenia przelewu");
28      }
29  }
```

Fragment kodu 10.6: Walidacja przejść statusu zgłoszenia

10.7 Obsługa garażu

Garaż udostępnia `GarageController` (`/api/garage`) wyłącznie użytkownikom zalogowanym. Serwis umożliwia listowanie, dodawanie, edycję i usuwanie pojazdów należących do bieżącego konta. Przy tworzeniu wymagane są co najmniej marka i model; pozostałe atrybuty techniczne i deklaracje (np. OC, przegląd, klatka) służą późniejszemu dopasowaniu do wymagań wydarzeń.

Ograniczenia spójności chronią otwarte procesy zapisów: przy aktywnym zgłoszeniu zablokowana jest zmiana kluczowych cech pojazdu, a usunięcie pojazdu nie jest możliwe, gdy bierze on udział w zgłoszeniach w stanach uniemożliwiających bezpieczne odpięcie (listing 10.7). Dzięki temu garaż pozostaje spójny z modułem zgłoszeń i regułami *carMatch*.

```
1   @Transactional
2   public void delete(String userId, String carId) {
3       Car car = carRepository.findByIdAndUser_Id(carId, userId)
4           .orElseThrow(() -> ApiException.notFound("Nie znaleziono auta"));
5
6       if (registrationRepository.existsByCar_IdAndStatusIn(
7               carId, OPEN_STATUSES)) {
8           throw ApiException.badRequest(
9               "Brak mozliwosci usuniecia auta ze wzgledu na otwarte zgloszenie");
10      }
11      registrationRepository.clearCarId(carId);
12      carRepository.delete(car);
13  }
```

Fragment kodu 10.7: Usuwanie pojazdu z blokadą przy otwartym zgłoszeniu

10.8 Panel administratora i organizatora

Panel administracyjny eksponuje `AdminController` pod prefiksem `/api/admin` z wymaganiem roli `ADMIN`. Udostępnia m.in. statystyki platformy, listę użytkowników, zmianę ról (z za-

bezpieczeniami przed usunięciem ostatniego administratora), kolejki wydarzeń do moderacji, zmianę statusów wydarzeń oraz rozstrzyganie wniosków o rolę organizatora (akceptacja skutkuje nadaniem roli i powiadomieniem e-mail).

Ścieżki organizatora obejmują `OrganizerController` (`/api/organizer`): podgląd własnego wniosku, złożenie wniosku o rolę oraz listę wydarzeń zarządzanych przez organizatora (administrator widzi pełny zbiór). Operacje tworzenia i edycji wydarzeń oraz zarządzania zgłoszeniami uczestników są realizowane przez zasoby wydarzeń i zgłoszeń z kontrolą dostępu opartą o role i własność zasobu. Taki podział utrzymuje czytelne granice RBAC bez dublowania całej logiki w jednym kontrolerze. Klasę panelu administratora przedstawia listing 10.8.

```
@RestController
@RequestMapping("/api/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    @GetMapping("/events/pending")
    public ResponseEntity<List<EventResponse>> pendingEvents() {
        return ResponseEntity.ok(adminService.pendingEvents());
    }

    @PatchMapping("/organizer-applications/{id}")
    public ResponseEntity<OrganizerApplicationResponse>
            updateOrganizerApplication(@PathVariable String id,
                @Valid @RequestBody
                OrganizerApplicationStatusUpdateRequest request) {
        return ResponseEntity.ok(
            adminService.updateOrganizerApplicationStatus(
                id, request.status()));
    }
}
```

Fragment kodu 10.8: Panel administratora — RBAC na poziomie klasy

10.9 Integracja z bazą danych

Trwałość danych zapewnia MySQL w połączeniu ze Spring Data JPA. Konfiguracja połączenia pochodzi ze zmiennych środowiskowych (w środowisku Compose hostem jest usługa kontenerowa bazy). Aplikacja działa z `spring.jpa.open-in-view=false`. Schemat jest utrzymywany trybem `ddl-auto` (domyślnie `update`); w projekcie nie zastosowano Flyway ani Liquibase — jest to świadome uproszczenie MVP, akceptowalne przy kontrolowanym środowisku demonstracyjnym, lecz wymagające ostrożności przy dalszym rozwoju produkcyjnym.

Dla list wydarzeń zastosowano cache Caffeine (krótki czas życia), invalidowany m.in. przy archiwizacji. Seed danych startowych (`DataInitializer`) zapewnia konta demonstracyjne, katalog torów oraz przykładowe treści ułatwiające prezentację ról. Kluczowe ustawienia JPA i cache pokazuje listing 10.9.

```yaml
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:mysql://localhost:3306/raceportal}
    username: ${DATABASE_USERNAME:raceportal}
    password: ${DATABASE_PASSWORD:raceportal}
  jpa:
    hibernate:
      ddl-auto: ${JPA_DDL_AUTO:update}
    open-in-view: false
  cache:
    type: caffeine
    cache-names: events
    caffeine:
      spec: maximumSize=500,expireAfterWrite=20s
```

Fragment kodu 10.9: Konfiguracja MySQL/JPA i cache Caffeine (`application.yml`)

10.10 Obsługa wyjątków i walidacja

Wejście API jest walidowane adnotacjami Bean Validation na rekordach DTO (`@Valid` w kontrolerach). Błędy reguł biznesowych i autoryzacji są reprezentowane przez `ApiException` z odpowiednim kodem HTTP (m.in. 400, 401, 403, 404, 409, 429).

`GlobalExceptionHandler` (`@RestControllerAdvice`) mapuje wyjątki na jednolite odpowiedzi JSON, w tym błędy walidacji pól, naruszenia integralności danych, odmowy dostępu oraz limity liczby prób (z nagłówkiem `Retry-After` przy 429). Dzięki temu klienci webowy i mobilny otrzymują przewidywalny kontrakt błędów, a szczegóły techniczne wyjątków nie są niekontrolowanie ujawniane użytkownikowi końcowemu. Fabryki `ApiException` oraz handler walidacji przedstawiają listingi 10.10 i 10.11.

```
public static ApiException badRequest(String message) {
    return new ApiException(HttpStatus.BAD_REQUEST, message);
}
public static ApiException unauthorized(String message) {
    return new ApiException(HttpStatus.UNAUTHORIZED, message);
}
public static ApiException forbidden(String message) {
    return new ApiException(HttpStatus.FORBIDDEN, message);
}
public static ApiException notFound(String message) {
    return new ApiException(HttpStatus.NOT_FOUND, message);
}
public static ApiException conflict(String message) {
    return new ApiException(HttpStatus.CONFLICT, message);
}
public static ApiException tooManyRequests(String message) {
    return new ApiException(HttpStatus.TOO_MANY_REQUESTS, message);
}
```

Fragment kodu 10.10: Fabryki wyjątków biznesowych API

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<ErrorResponse> handleApiException(ApiException ex) {
        return ResponseEntity.status(ex.getStatus())
            .body(new ErrorResponse(ex.getMessage(), ex.getDetails()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(fe ->
            fieldErrors.put(fe.getField(), fe.getDefaultMessage()));
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse("Nieprawidlowe dane", fieldErrors));
    }
}
```

Fragment kodu 10.11: Globalna obsługa `ApiException` i błędów `@Valid`

Rozdział 11

Szczegóły implementacji frontendu webowego

11.1 Struktura aplikacji webowej

Warstwa klientowska systemu RacePortal została zaimplementowana jako zaawansowana aplikacja jednostronicowa (SPA) oparta o bibliotekę **React 18** oraz język **TypeScript**, wykorzystująca bundler **Vite** oraz stylowanie za pomocą **Tailwind CSS** i komponentów Radix UI. Architektura katalogów w module `web/src/app` została zorganizowana modularnie, co zapewnia czytelność i łatwość utrzymania kodu:

- `pages/` — komponenty widoków ekranowych reprezentujące poszczególne podstrony aplikacji.
- `components/` — reużywalne elementy interfejsu (formularze, karty, modale, elementy nawigacyjne).
- `context/` — zarządzanie stanem globalnym, w tym główny kontekst autoryzacji (`AuthContext`) kontrolujący sesję użytkownika.
- `lib/` — warstwa logiki sieciowej (`api.ts`), typowania oraz konwersji danych zgodna ze wspólnym pakietem typów API.

Poniższy kod realizuje dynamiczne routowanie aplikacji przy użyciu `createBrowserRouter` z React Router v7, definiując układ bazowy (`RootLayout`) oraz chronione trasy dostępu dla poszczególnych ról systemowych.

```
1  export const router = createBrowserRouter([
2    {
3      path: "/",
4      Component: RootLayout,
5      children: [
6        { index: true, Component: HomePage },
7        { path: "login", Component: LoginPage },
8        { path: "wydarzenia", Component: EventsPage },
```

```
9         { path: "wydarzenia/:id", Component: EventDetailPage },
10        { path: "dashboard", Component: ProtectedDashboard },
11        { path: "garaz", Component: ProtectedGarage },
12        { path: "organizer", Component: ProtectedOrganizer },
13        { path: "admin", Component: ProtectedAdmin },
14      ],
15    },
16  ]);
```

Fragment kodu 11.1: Fragment drzewa tras React Router (katalog, auth, panele) — warstwa webowa.

11.2 Strona główna i katalog wydarzeń

Strona główna integruje główny katalog wydarzeń motorsportowych, oferując użytkownikom trzy spójne widoki z zachowaniem tych samych filtrów: **listę chronologiczną**, **interaktywną mapę** oraz **kalendarz**.

- W widoku listy zaimplementowano optymalizację w postaci mechanizmu *debounce* dla filtrów, co zapobiega wysyłaniu nadmiarowych zapytań HTTP do backendu podczas wpisywania fraz wyszukiwania.
- Użytkownik ma do dyspozycji funkcję „Wyczyść filtry” oraz paginację listy zsynchronizowaną z markerami przeglądowymi na mapie.

11.3 Widok szczegółów wydarzenia

Podstrona szczegółów dostarcza użytkownikowi pełnych informacji o wybranej imprezie motorsportowej:

- Wyświetla dokładny opis, dane organizatora oraz rygorystyczne wymagania techniczne stawiane pojazdom i uczestnikom.
- Integruje moduł szybkiego zapisu na wydarzenie, który weryfikuje posiadane przez kierowcę auta w garażu pod kątem reguł dopasowania kategorii (`carMatch`).
- Zawiera mapę z wyznaczaniem trasy dojazdu do lokalizacji obiektu z wykorzystaniem routingu OSRM [11].

11.4 Logowanie i rejestracja

Moduł autoryzacji opiera się na kontrolowanych formularzach frontendowych, wspierających procesy logowania, rejestracji oraz odzyskiwania zapomnianego hasła. Bezpieczeństwo sesji

gwarantują ciasteczka typu httpOnly (`rp_at`, `rp_rt`) przesyłane z flagą `credentials: "include"`, co całkowicie eliminuje podatności związane z przechowywaniem tokenów JWT w lokalnej pamięci przeglądarki (`localStorage`).

Poniższy kod implementuje asynchroniczne pobieranie danych o zalogowanym użytkowniku przy użyciu mechanizmu optymalizacyjnego `deferUntilIdle`, dbając o to, by zapytanie nie blokowało renderowania głównego wątku aplikacji.

```
useEffect(() => {
  clearLegacyToken();
  return deferUntilIdle(() => {
    void (async () => {
      try {
        const me = await api.get<User>("/api/auth/me");
        persistAuth(me);
      } catch {
        persistAuth(null);
      } finally {
        setIsLoading(false);
      }
    })();
  }, 2000);
}, []);
```

Fragment kodu 11.2: Wczytywanie sesji w `AuthContext` (httpOnly cookies).

11.5 Dashboard użytkownika

Panel użytkownika (Dashboard) stanowi centralne miejsce zarządzania aktywnością zalogowanego kierowcy w systemie. Prezentuje on przejrzyste podsumowanie aktualnych zgłoszeń użytkownika na nadchodzące wydarzenia motorsportowe, umożliwiając błyskawiczny podgląd ich statusów (oczekujące, zatwierdzone, anulowane) oraz dostęp do szczegółów rezerwacji.

11.6 Widok garażu

Moduł wirtualnego garażu pozwala kierowcom na pełne zarządzanie flotą własnych pojazdów startowych.

- Interfejs dostarcza formularz dodawania nowego pojazdu z rygorystyczną walidacją pól wymaganych (takich jak marka, model, specyfikacja techniczna, masa czy dopasowanie opon).
- Użytkownik ma możliwość edycji danych oraz usuwania aut ze swojego profilu, przy czym system uwzględnia zabezpieczenia biznesowe (np. blokadę usunięcia pojazdu, który został powiązany z aktywnym, zatwierdzonym zgłoszeniem na zawody).

11.7 Panel organizatora

Panel organizatora udostępnia dedykowane narzędzia dla podmiotów publikujących imprezy motorsportowe. Umożliwia on:

- Tworzenie oraz edycję wydarzeń w jednym spójnym formularzu z wykorzystaniem wbudowanych presetów torów.
- Podgląd napływających zgłoszeń od kierowców oraz zarządzanie ich statusami (np. akceptacja uczestnictwa, odrzucenie lub zmiana statusu płatności).

11.8 Panel administratora

Panel administratora zapewnia operatorom platformy narzędzia najwyższego poziomu do nadzoru nad treścią oraz strukturą uprawnień w systemie. Obejmuje on:

- Moderację nadesłanych wydarzeń oraz rozpatrywanie wniosków użytkowników ubiegających się o uzyskanie uprawnień organizatora (dedykowana ścieżka formularza pod adresem `/zostan-organizatorem`).
- Zarządzanie globalnymi rolami kont oraz podgląd systemowych zgłoszeń błędów.

11.9 Mapa i elementy lokalizacyjne

Warstwa mapowa opiera się na integracji biblioteki Leaflet [12] z kafelkami CARTO / OpenStreetMap. Umożliwia ona interaktywną prezentację punktów zainteresowania, selekcję współrzędnych geograficznych w formularzach wydarzeń oraz płatną lub opartą o OSRM [11] integrację routingu trasy dojazdu dla kierowców (proxy `/api/maps/route`).

11.10 Responsywność i doświadczenie użytkownika

Interfejs aplikacji zaprojektowano w koncepcji *mobile-first*. Układy siatek (grid, flexbox) w Tailwind CSS dynamicznie dostosowują się do ekranów smartfonów, tabletów oraz monitorów stacjonarnych. Spójność wizualną oparto o ciemny motyw motorsportowy z wyrazistym akcentem `-race-accent`, optymalizując czytelność danych i zapewniając wysoką użyteczność (UX).

Rozdział 12

Szczegóły implementacji aplikacji mobilnej

Rozdział opisuje implementację klienta mobilnego RacePortal. Aplikacja powstała w technologii Expo (React Native) i pełni rolę publicznej wizytówki katalogu: umożliwia przeglądanie wydarzeń na telefonie, natomiast zapis udziału oraz funkcje konta realizowane są w wersji webowej.

12.1 Założenia aplikacji mobilnej

Przyjęto świadome ograniczenie zakresu względem klienta webowego. Aplikacja mobilna ma przede wszystkim umożliwić przeglądanie nadchodzących wydarzeń motorsportowych — w postaci listy, mapy, kalendarza oraz ekranu szczegółów — a nie stanowić drugiej, pełnej aplikacji z logowaniem i garażem.

Założenia wynikające z tego podziału:

- brak uwierzytelniania i przechowywania tokenów JWT w aplikacji mobilnej;
- wyłącznie publiczne operacje odczytu katalogu;
- konwersja zapisu przez otwarcie odpowiedniej strony wydarzenia w przeglądarce / PWA;
- wspólny kontrakt danych z API (te same wydarzenia i filtry co na webie);
- brak publikacji w sklepach App Store / Google Play w ramach MVP (uruchomienie przez Expo / emulator).

We wcześniejszej fazie projektu rozważano szerszą parity z webem; ostatecznie zakres zawężono do wizytówki, aby uniknąć dublowania logiki zapisów i utrzymać jeden główny kanał transakcyjny.

12.2 Architektura rozwiązania mobilnego

Rozwiązanie oparto o Expo SDK oraz React Native. Warstwa prezentacji korzysta z React Navigation (native stack) ograniczonego do przepływu wydarzeń. Motyw wizualny (ciemne tło, akcent marki) oraz lokalizacja interfejsu (PL/EN) są dostarczane przez konteksty aplikacji.

Klient HTTP ograniczono do minimum potrzebnego wizytówce: wykonuje wyłącznie publiczne żądania `GET` do API RacePortal, bez nagłówka autoryzacji. Adres API i adres wersji webowej (używany przy CTA zapisu) konfigurowane są zmiennymi środowiskowymi Expo, z domyślnymi wartościami dopasowanymi do środowiska deweloperskiego (m.in. inny host dla emulatora Androida niż dla symulatora iOS).

Aplikacja mobilna **nie** wchodzi w skład Docker Compose — uruchamiana jest osobno obok kontenerów web/API/MySQL.

12.3 Lista wydarzeń i widok szczegółów

Ekran listy pobiera wydarzenia z publicznego endpointu katalogu i udostępnia trzy tryby prezentacji: lista, kalendarz oraz mapa (markery). Filtry publiczne obejmują m.in. wyszukiwanie tekstowe, kategorię, województwo, miasto, tor, zakres dat oraz oznaczenie wydarzeń płatnych / bezpłatnych. Zestawy lokalizacji (województwa, tory) są spójne z listami używanymi po stronie weba, aby obie aplikacje operowały na tych samych słownikach.

Dobór zakresu dat korzysta z natywnego komponentu wyboru daty. Wygląd i zachowanie różnią się między platformami: na iOS stosowany jest tryb `inline`, na Androidzie — systemowy dialog (`default`). Po wyborze daty na Androidzie selektor jest zamykany od razu (listing 12.1).

```
const onDatePicked = (event, selectedDate) => {
  if (Platform.OS === "android") setDatePickerTarget(null);
  if (event.type !== "set" || !selectedDate || !datePickerTarget) return;
  const value = formatDate(selectedDate);
  if (datePickerTarget === "from") setDateFrom(value);
  if (datePickerTarget === "to") setDateTo(value);
};

{datePickerTarget && (Platform.OS === "android" || Platform.OS === "ios") ? (
  <DateTimePicker
    value={currentPickerDate}
    mode="date"
    display={Platform.OS === "ios" ? "inline" : "default"}
    onChange={onDatePicked}
  />
) : null}
```

Fragment kodu 12.1: Wybór daty filtra z rozróżnieniem iOS / Android

Widok szczegółów pobiera pojedyncze wydarzenie po identyfikatorze i prezentuje podstawowe metadane (nazwa, kategoria, data, lokalizacja, opis, multimedia). Z poziomu detalu użytkownik może otworzyć lokalizację w zewnętrznej aplikacji map oraz przejść do zapisu na stronie webowej. Otwarcie map realizuje systemowe `Linking` z zapytaniem zawierającym tor, miasto i współrzędne (listing 12.2).

```
const openMaps = () => {
  if (!event?.lat || !event?.lng) return;
  const q = encodeURIComponent(
    `${event.track}, ${event.city} ${event.lat},${event.lng}`
  );
  void Linking.openURL(
    `https://www.google.com/maps/search/?api=1&query=${q}`
  );
};
```

Fragment kodu 12.2: Otwarcie lokalizacji wydarzenia w zewnętrznej aplikacji map

12.4 Nawigacja i interfejs użytkownika

Nawigacja ogranicza się do jednego stosu ekranów: lista wydarzeń oraz detal wybranego wydarzenia. Nie zastosowano dolnych kart (tabów) typowych dla pełnego klienta, ponieważ poza katalogiem nie ma w mobilce osobnych obszarów konta, garażu ani paneli ról. Ilustruje to listing 12.3: zarejestrowane są wyłącznie ekrany katalogu.

```
function EventsNavigator() {
  const { accentColor } = useTheme();
  const { t } = useLocale();
  return (
    <EventsStack.Navigator
      screenOptions={{
        headerStyle: { backgroundColor: colors.card },
        headerTintColor: accentColor,
        headerTitleStyle: { color: colors.text, fontWeight: "800" },
      }}
    >
      <EventsStack.Screen
        name="EventsList"
        component={EventsScreen}
        options={{ headerShown: false }}
      />
      <EventsStack.Screen
        name="EventDetail"
        component={EventDetailScreen}
        options={{ title: t("events.details") }}
      />
    </EventsStack.Navigator>
  );
}
```

Fragment kodu 12.3: Stos nawigacji ograniczony do listy i detalu wydarzenia

Interfejs utrzymuje ciemny motyw motorsportowy spójny z identyfikacją RacePortal. Elementy wspólne (nagłówki, przyciski, stany ładowania i braku danych) wydzielono do prostych komponentów UI, aby zachować jednolity wygląd obu ekranów.

12.5 Integracja z API

Integracja obejmuje wyłącznie publiczne odczyty potrzebne katalogowi, w szczególności:

- listę wydarzeń (z filtrami i podziałem na strony),
- markery mapy,
- detal wydarzenia,
- opcjonalnie sprawdzenie dostępności API (health) w scenariuszach diagnostycznych.

Ścieżki względne multimediów (np. uploady) są uzupełniane o adres bazowy API, ponieważ klient natywny — w odróżnieniu od aplikacji webowej serwowanej z tego samego hosta co API — nie rozwiązuje automatycznie ścieżek względnych zasobów.

Konfiguracja hosta uwzględnia typowe różnice środowisk: lokalny host na symulatorze iOS, specjalny adres bramy emulatora Androida (`10.0.2.2`) oraz możliwość wskazania adresu LAN / produkcji przez zmienne Expo. Fragment z listingu 12.4 pokazuje domyślne wyznaczanie adresu API (analogiczna logika dotyczy adresu wersji webowej używanego przy CTA).

```
export const API_URL = (() => {
  if (process.env.EXPO_PUBLIC_API_URL) {
    return process.env.EXPO_PUBLIC_API_URL.replace(/\/$/, "");
  }
  const expoGoHost = resolveExpoGoHost();
  if (expoGoHost && expoGoHost !== "localhost"
      && expoGoHost !== "127.0.0.1") {
    return `http://${expoGoHost}:4000`;
  }
  return Platform.OS === "android"
    ? "http://10.0.2.2:4000"
    : "http://127.0.0.1:4000";
})();
```

Fragment kodu 12.4: Wyznaczanie bazowego adresu API dla klienta mobilnego

12.6 Przekierowanie do wersji webowej

Zapis na wydarzenie nie jest realizowany w aplikacji mobilnej. Na ekranie detalu umieszczono przycisk CTA, który buduje adres wersji webowej wydarzenia i otwiera go systemowym mechanizmem łączy zewnętrznych (`Linking`). Użytkownik trafia tym samym do pełnego flow zapisu na webie (konto, garaż, statusy zgłoszenia, ewentualny dowód płatności).

Takie rozwiązanie utrzymuje jedną implementację reguł biznesowych zapisów i zmniejsza ryzyko rozjazdu mobile $\leftrightarrow$ web.

12.7 Ograniczenia wersji mobilnej

Wersja mobilna MVP nie obejmuje:

- logowania, rejestracji ani zarządzania profilem;
- garażu pojazdów i składania zgłoszeń bezpośrednio z aplikacji;
- paneli organizatora i administratora;
- archiwum, wyników zawodów ani funkcji społecznościowych;
- powiadomień push oraz publikacji w sklepach aplikacji.

Ograniczeniem wynikającym z przyjętego modelu CTA jest zależność od dostępności wersji webowej w momencie zapisu. Jednocześnie ograniczenie to jest zamierzone: mobilka pozostaje lekkim kanałem przeglądania katalogu, a kompletny produkt transakcyjny skupia się na webie (w tym PWA).

Rozdział 13

Bezpieczeństwo systemu

13.1 Założenia bezpieczeństwa

Ochrona kont i danych zgłoszeń na poziomie ASVS L1 adekwatnym do MVP demonstracyjnego.

13.2 Klasyfikacja danych

Dane konta i profilu; dane pojazdów; dane zgłoszeń; treści wydarzeń (częściowo publiczne).

13.3 Uwierzytelnianie użytkowników

Hasło + JWT (access/refresh); opcjonalnie Google; brak klasycznej sesji serwerowej w pamięci — stan odświeżany cookie refresh (`rp_rt`).

13.4 Autoryzacja i kontrola dostępu RBAC

Role USER / ORGANIZER / ADMIN egzekwowane w API i bramkach UI.

13.5 Wykorzystanie tokenów JWT

W warstwie webowej access token jest w ciasteczku httpOnly `rp_at` (JavaScript go nie odczytuje); `localStorage` nie przechowuje JWT. Filtr API odczytuje cookie albo nagłówek `Authorization: Bearer` (m.in. testy).

13.6 Walidacja danych wejściowych

Walidacja DTO po stronie API; walidacja formularzy UI; komunikaty pól `details`.

13.7 Ochrona zasobów użytkowników

Dostęp do garażu i zgłoszeń wyłącznie właściciela (oraz uprawnionych ról w panelach).

13.8 Bezpieczeństwo komunikacji

W dev HTTP lokalne; w prod TLS terminowany na Cloudflare Tunnel przed nginx; CSP / X-Frame-Options na nginx.

13.9 Analiza zagrożeń

Uwierzytelnianie, IDOR, XSS, CSRF w kontekście SPA, brute-force logowania (rate limit).

13.10 Analiza ryzyka

Ryzyko akceptowalne dla środowiska demo; produkcja wymaga twardego sekretu JWT, HTTPS i polityki haseł zsynchronizowanej backend↔UI.

13.11 Ograniczenia przyjętych zabezpieczeń

Brak 2FA; Google OAuth opcjonalny; brak WAF; publiczny OSRM bez SLA.

Rozdział 14

Testowanie i weryfikacja systemu

14.1 Strategia testowania

Piramida: unit (api-types, mobile client) → integracyjne API (MockMvc) → E2E Playwright (web + preview mobile).

14.2 Środowisko testowe

Docker Compose + opcjonalnie Expo web `:8082`; dokumentacja `docs/testy/TESTY.md`.

14.3 Testy jednostkowe

Vitest: `packages/api-types`, `mobile/tests`.

14.4 Testy integracyjne backendu

JUnit / MockMvc — scenariusze auth, eventy, garaż, zgłoszenia, RBAC, trasa.

14.5 Testy End-to-End aplikacji webowej

Playwright: ścieżki katalogu, auth, garaż, role (viewport desktop/mobile).

14.6 Testy aplikacji mobilnej

Unit URL API/WEB; E2E gość (lista, widoki, detal, CTA) na Expo web.

14.7 Testy mechanizmów bezpieczeństwa

Scenariusze braku dostępu do paneli obcych ról; walidacja wejścia.

14.8 Automatyzacja uruchamiania testów

Skrypty npm / `scripts/`; raporty w `docs/testy/wyniki/`.

14.9 Analiza wyników testów

Werdykty przebiegów unit/API/E2E odnotowywane są w dokumentacji wyników (`docs/testy/wyniki/`); regresie łapane przed prezentacją.

Osobno, 25.10.2026, wykonano pomiary obciążenia Grafana k6 [13] na Oracle VM (ruch na originie `localhost`, za Cloudflare Tunnel). Scenariusze: ramp/constant arrival-rate na `GET /api/health`, `GET /` (nginx) oraz `GET /api/events?page=0&size=10`. Skrótowo: health i front static utrzymują ok. 2000 RPS w strefie komfortu; lista wydarzeń z MySQL — ok. 150–200 RPS (przy 50 RPS: 0% błędów, p95 ok. 4,5 ms). Pełne tabele i interpretacja: `docs/testy/wyniki/k6-vm-rps.md` [8].

14.10 Ograniczenia procesu testowania

Brak pełnych testów natywnych (Detox/Maestro). Nie wykonano formalnego scenariusza odbiorczego KP/DZW „ok. 10 000 rekordów w katalogu / 50 RPS” — w raporcie k6 nie ma zapełnienia bazy do 10k; są pomiary RPS na istniejących danych. Dodatkowo k6 działał na tej samej VM co stack (konkurencja o 2 OCPU), więc wynik to dolna granica pojemności originu; szczegóły także w podsumowaniu [8].

Rozdział 15

Prezentacja systemu

15.1 Prezentacja głównych funkcjonalności

Demonstracja katalogu (3 widoki), zapisu kierowcy z `carMatch`, paneli org/admin oraz mobile CTA — zgodnie z `docs/scenariusz-prezentacji.md`.

15.2 Scenariusz użycia z perspektywy kierowcy

Login seed → garaż → filtr/katalog → zapis → dashboard statusów.

15.3 Scenariusz użycia z perspektywy organizatora

Login org → wydarzenie (presety torów) → zgłoszenia → zmiana statusu → podgląd maila na skrzynce (Mailbux).

15.4 Scenariusz użycia z perspektywy administratora

Login admin → moderacja wydarzenia / wniosku organizatora / ról.

15.5 Podsumowanie działania systemu

System realizuje spójny obieg informacji o wydarzeniach i zapisach w środowisku uruchamialnym lokalnie oraz pod adresem produkcyjnym za Cloudflare Tunnel.

Rozdział 16

Nakład pracy

16.1 Liczba godzin przeznaczona na realizację projektu

Nakład sumaryczny (osobogodziny): **[do uzupełnienia przez zespół]**.

16.2 Podział zadań w zespole

Skrót obszarów — rozdz. 17. Szczegółowa tabela godzin per osoba: **[do uzupełnienia przez zespół]**.

16.3 Narzędzia wspierające zarządzanie pracą

Git/GitHub, dokumentacja Markdown w `docs/`, Docker, Cursor/IDE, Playwright, Figma Make, Cloudflare Dashboard; skład pracy: LaTeX [14].

16.4 Problemy organizacyjne i ich rozwiązanie

M.in. rozrost zakresu mobile — decyzja o wizytówce; luki Spec — rejestr `spec-conformity.md`; stabilizacja testów względem seedu/archiwizacji; uproszczenie hostingu (Oracle + Tunnel zamiast GCP Pages).

Rozdział 17

Wkład własny

Rozdział opisuje, kto odpowiadał za które obszary prac projektowych i implementacyjnych (imię i nazwisko). Podział odzwierciedla faktyczny udział zespołu; największy wkład merytoryczny i projektowy w warstwie produktu/UI wniosła Natalia Otrombke.

17.1 Natalia Otrombke

- projekt UI/UX w Figma Make (**Advanced UI/UX for RACEPORTAL**) — ekrany katalogu, auth, dashboard, paneli;
- przeniesienie i dopracowanie frontendu React (motyw, layout, spójność marki, kluczowe ścieżki UX);
- udział w definicji flow użytkownika (gość $\rightarrow$ kierowca $\rightarrow$ organizator) oraz w materiałach koncepcyjnych ról.

17.2 Oliwier Kasprowicz

- prace analityczne i dokumentacyjne (wymagania, prezentacje zespołowe);
- wsparcie przy diagramach i scenariuszach użycia;
- udział w przeglądach zakresu MVP i odbiorze funkcjonalności.

17.3 Michał Gutowski

- diagramy i materiały projektowe (m.in. rich picture, przepływy);
- wsparcie dokumentacji i testów scenariuszy prezentacji;
- udział w integracji i weryfikacji ścieżek użytkownika.

17.4 Miłosz Parkitny

- modelowanie i diagramy (m.in. przypadki użycia, podział funkcjonalności, model danych);
- wsparcie analizy interesariuszy i harmonogramów;
- udział w pracach implementacyjnych / poprawkach w repozytorium.

17.5 Wojciech Wronisz

- backend Spring Boot (API, JWT/RBAC, encje, seed, maile);
- Docker Compose, wdrożenie produkcyjne Oracle + **Cloudflare Tunnel**, Mailbux, PWA;
- dokumentacja uruchomienia i odbioru (`docs/`), testy API / E2E, integracja web $\leftrightarrow$ API.

Rozdział 18

Podsumowanie

18.1 Największe wyzwania i sposób ich rozwiązania

Podczas realizacji i wdrożenia produkcyjnego systemu RacePortal zespół napotkał szereg istotnych wyzwań inżynierskich, zwłaszcza w obszarze infrastruktury, konfiguracji bezpiecznego tunelowania oraz zabezpieczeń sieciowych. Do najważniejszych z nich należały:

- **Wyzwania związane z wdrożeniem (deployment) i architekturą chmurową:** Początkowe próby rozproszenia aplikacji (np. frontend hostowany na zewnętrznych platformach typu Pages, a backend na maszynie wirtualnej) generowały problemy z konfiguracją polityk CORS, routingu oraz spójności środowiskowej. Wyzwaniem było również zapewnienie prostoty uruchomienia produkcyjnego na darmowej maszynie wirtualnej (Oracle Cloud Always Free ARM). *Sposób rozwiązania:* Zdecydowano się na pełną konteneryzację całego ekosystemu za pomocą narzędzia Docker Compose (`web`, `api`, `mysql`, `backup`) oraz sprowadzenie infrastruktury produkcyjnej do jednej, spójnej instancji obliczeniowej zarządzanej kodem konfiguracyjnym.

- **Konfiguracja bezpiecznego tunelowania w Cloudflare:** Wystawienie lokalnie skonteneryzowanej aplikacji na publiczny internet przy jednoczesnym zachowaniu wysokich standardów bezpieczeństwa i szyfrowania bez konieczności skomplikowanego zarządzania certyfikatami SSL bezpośrednio na serwerze stanowiło duże wyzwanie sieciowe. *Sposób rozwiązania:* Zastosowano technologię **Cloudflare Tunnel** (`cloudflared`). Tunel bezpiecznie spina maszynę wirtualną z brzegiem sieci Cloudflare, przejmując całkowitą obsługę rekordów DNS, terminację protokołu TLS (HTTPS) oraz ochronę przed atakami DDoS (WAF). Dzięki temu ruch do lokalnego serwera Nginx (działającego na porcie `:8081`) jest w pełni szyfrowany i odizolowany od bezpośrednich skanów portów z zewnątrz.

- **Zabezpieczenia strony pod kątem cybersecurity i ochrony zasobów:** Projektując platformę obsługującą dane użytkowników, konta ról oraz pojazdy, należało zabezpieczyć

system przed typowymi zagrożeniami webowymi (takimi jak próby przejęcia sesji, ataki typu brute-force na panele logowania czy nieuprawniony dostęp do zasobów innych użytkowników — IDOR). *Sposób rozwiązania:* Wdrożono bezstanowe uwierzytelnianie oparte o tokeny JSON Web Token (JWT) przesyłane w bezpiecznych nagłówkach oraz rygorystyczną kontrolę dostępu opartą o role (RBAC) na poziomie kontrolerów API. Dodatkowo kluczowe ścieżki uwierzytelniania zabezpieczono mechanizmami limitowania częstotliwości zapytań (Rate Limiting), a na warstwie serwera proxy (Nginx) skonfigurowano restrykcyjne nagłówki bezpieczeństwa, w tym politykę ochrony treści (CSP) oraz nagłówki zapobiegające osadzaniu strony w ramkach (X-Frame-Options).

18.2 Zmiany i adaptacje w trakcie realizacji

Przejście mobile z pełnej równoważności funkcji na wizytówkę; doprecyzowanie formularzy; wspólny pakiet typów; migracja z eksperymentów GCP/Pages na Oracle + Cloudflare Tunnel.

18.3 Ograniczenia końcowej wersji systemu

Oddawana wersja RacePortal domyka założony zakres MVP: publiczny katalog wydarzeń, konta i role, wirtualny garaż, proces zgłoszeń ze statusami, panele organizatora i administratora, powiadomienia e-mail, lokalny upload mediów oraz mobilną wizytówkę katalogu. Jednocześnie praca inżynierska wymaga jasnego wskazania granic rozwiązania — zarówno tych wynikających ze świadomego zawężenia zakresu, jak i tych wynikających z przyjętego modelu wdrożenia demonstracyjnego. Opis poniżej jest spójny z kartą projektu oraz z podrozdziałem 1.1; nie stanowi katalogu usterek, lecz zestaw ograniczeń, których zespół nie deklarował jako zrealizowanych.

Najistotniejszym ograniczeniem biznesowym pozostaje brak natywnej bramki płatności online (PSP). System umożliwia oznaczanie statusów opłat i dołączanie potwierdzenia przelewu, lecz nie pobiera składek startowych, nie rozlicza zwrotów i nie prowadzi księgowości wydarzenia. Integracja PSP została odłożona ze względu na koszty onboardingu, wymogi identyfikacji płatnika oraz odpowiedzialność prawną nieproporcjonalną do celów akademickiego prototypu.

W obszarze treści i plików końcowa wersja wykracza poza wczesne założenie „same referencje URL”. Aplikacja webowa oraz API umożliwiają lokalny upload obrazów (JPG, PNG, WebP) i dokumentów PDF z limitem rozmiaru oraz weryfikacją typu po sygnaturze pliku. Mechanizm ten obsługuje m.in. zdjęcia pojazdów w garażu, grafikę wydarzenia, regulaminy PDF oraz opcjonalny dowód płatności przy zgłoszeniu; równolegle nadal dopuszczalne jest wskazanie zewnętrznego adresu http(s). Ograniczeniem pozostaje natomiast warstwa infrastrukturalna przechowywania: pliki trafiają na dysk serwera aplikacji, bez zewnętrznego object storage, bez skanowania antywirusowego, bez polityk retencji i bez podpisanych odnośników o czasowo

ograniczonym dostępie. Nie zaimplementowano także galerii albumów po imprezie ani uploadu mediów z klienta mobilnego, który w MVP pełni wyłącznie rolę wizytówki.

Komunikacja z użytkownikiem opiera się na poczcie elektronicznej (potwierdzenia rejestracji, weryfikacja konta, zmiany statusów zgłoszeń) oraz uzupełniających powiadomieniach w aplikacji webowej. Brakuje natywnych powiadomień push oraz narzędzi do kampanii marketingowych. Świadomie zawężono również zakres aplikacji mobilnej: klient Expo udostępnia przegląd katalogu (lista, mapa, kalendarz) z przekierowaniem do serwisu webowego w celu zapisu, lecz nie obejmuje logowania, garażu, zgłoszeń ani paneli ról i nie został opublikowany w sklepach App Store / Google Play. Decyzja ta pozwoliła domknąć pełną ścieżkę zapisu po stronie weba i PWA zamiast dublować ten sam zakres na dwóch klientach.

Zależności zewnętrzne dotyczą przede wszystkim map i tras. Domyślne wyznaczanie trasy korzysta z publicznego OSRM (z możliwością Google Maps API przy kluczu), co oznacza brak gwarancji dostępności i SLA poza kontrolą zespołu. Katalog torów jest seedowany i rozszerzalny, lecz nie pretenduje do roli oficjalnego, kompletnego rejestru obiektów motorsportowych w Polsce. Podobnie kompletność oferty wydarzeń zależy od aktywności organizatorów i moderacji administratora — platforma nie scrapuje automatycznie zewnętrznych źródeł informacji.

W warstwie bezpieczeństwa przyjęto standard adekwatny do MVP demonstracyjnego: uwierzytelnianie JWT w ciasteczkach HTTP-only, kontrola dostępu RBAC, ograniczanie liczby prób, walidacja wejścia oraz terminacja TLS/HSTS na Cloudflare Tunnel przed aplikacją. Poza zakresem pozostają m.in. pełny program testów penetracyjnych, zaawansowany WAF, dedykowany menedżer sekretów oraz własny certyfikat TLS utrzymywany bezpośrednio na hoście aplikacji. Są to ograniczenia dojrzałości operacyjnej, a nie brak podstawowych mechanizmów ochrony kont i zasobów opisanych w rozdziale poświęconym bezpieczeństwu.

18.3.1 Ograniczenia wydajnościowe (RPS)

W wymaganiach jakościowych MVP zapisano **formalny** test obciążenia: **ok. 10 000 rekordów** w katalogu oraz **ok. 50 RPS** przy akceptowalnym P95 odpowiedzi list/detalu. W oddawanej wersji należy rozróżnić dwie warstwy:

- zastosowano lokalne ograniczenia skutków (paginacja listy, osobne markery mapy, indeksy / cache tam gdzie uzasadnione);
- **nie wykonano** dokładnego scenariusza 10k rekordów / 50 RPS jako formalnego artefaktu odbiorczego KP/DZW — w raporcie k6 nie ma zapełnienia katalogu do 10k;
- wykonano natomiast **pomiary Grafana k6** [13] na Oracle VM (25.10.2026, ruch lokalny na originie; k6 współdzielił CPU ze stackiem). Dla `GET /api/events` komfort to ok. 150–200 RPS (przy 50 RPS: 0% błędów, p95 ok. 4,5 ms); health i nginx —

rzędu 2k RPS. Wąskie gardło to endpointy z bazą. Szczegóły: `docs/testy/wyniki/k6-vm-rps.md` [8].

Ograniczenie formalne (brak scenariusza z 10k rekordami) jest świadome: priorytetem było domknięcie podstawowej ścieżki funkcjonalnej, bezpieczeństwa (JWT/RBAC, cookies httpOnly) i uruchamialności. Uzupełnienie formalnego testu 10k/50 RPS pozostaje w backlogu jakościowym.

18.4 Możliwości dalszego rozwoju

Mimo że wersja MVP systemu RacePortal w pełni realizuje założone cele funkcjonalne i procesowe, otwiera ona szerokie perspektywy na dalszą rozbudowę oraz komercjalizację platformy. W oparciu o zebrane doświadczenia, opinie użytkowników oraz analizę braków obecnej wersji, zidentyfikowano następujące kierunki rozwoju:

- **Integracja z profesjonalnymi bramkami płatności (PSP):** Wprowadzenie automatycznych płatności online (np. przelewy natychmiastowe, BLIK, karty płatnicze) zamiast obecnego mechanizmu weryfikacji na podstawie przesyłanego adresu URL dowodu wpłaty. Umożliwi to pełną automatyzację rozliczeń finansowych między kierowcami a organizatorami oraz natychmiastowe potwierdzanie opłacenia wpisowego.
- **Autorski pomiar czasowy i wyniki na żywo:** Wdrożenie dedykowanego modułu pomiaru czasu dla organizatorów z możliwością synchronizacji urządzeń pomiarowych bezpośrednio z aplikacją, co umożliwi uczestnikom oraz kibicom podgląd oficjalnych wyników zawodów na żywo.
- **Zaawansowane zarządzanie listami rezerwowymi i waitlistą:** Rozbudowa logiki biznesowej zgłoszeń o automatyczne zarządzanie kolejką rezerwową (*waitlist*). W przypadku rezygnacji jednego z uczestników, system mógłby automatycznie powiadamiać i przesuwać na listę główną kolejnego kierowcę z listy rezerwowej według zdefiniowanych kryteriów.
- **Pełnoprawna aplikacja mobilna i publikacja w sklepach:** Rozszerzenie obecnej wizytówkowej aplikacji mobilnej (Expo) o pełną funkcjonalność konta użytkownika, w tym mobilny wirtualny garaż, składanie zgłoszeń oraz bezpośredni wgląd do statusów, a także przygotowanie aplikacji do oficjalnej dystrybucji w sklepach Google Play oraz App Store.
- **Podstrona Auta:** Stworzenie dedykowanej, publicznej strony każdego samochodu w systemie, prezentującej jego szczegółowe dane techniczne, galerię zdjęć oraz pełną historię startów w ubiegłych i nadchodzących wydarzeniach motorsportowych.

- **Podstrona Kierowcy:** Implementacja publicznego profilu zawodnika, który w jednym miejscu agreguje jego osiągnięcia sportowe, zestawienie posiadanych aut z wirtualnego garażu oraz odnośniki do mediów społecznościowych.
- **Podstrona Obiektów Motorsportowych:** Rozbudowa systemu o kompletny katalog wszystkich torów i obiektów w Polsce, zawierający ich opisy, parametry techniczne oraz precyzyjną lokalizację na zintegrowanej mapie.
- **Podstrona Organizatora:** Wprowadzenie dedykowanego profilu publicznego dla klubów oraz organizatorów wydarzeń, prezentującego ich historię, opis działalności oraz zestawienie organizowanych przez nich imprez motorsportowych.

18.5 Wnioski końcowe

RacePortal to działające MVP katalogu wydarzeń motorsportowych. System pozwala przeglądać imprezy, prowadzić garaż, składać zgłoszenia oraz obsługiwać role kierowcy, organizatora i administratora. Całość uruchamia się w Docker Compose i nadaje się do prezentacji dyplomowej.

Projekt ma też jasne granice. Nie ma płatności online — rozliczenie wpisowego zostaje po stronie organizatora. Aplikacja mobilna pokazuje głównie katalog i kieruje do zapisu w wersji webowej; nie ma w niej logowania, garażu ani publikacji w sklepach. Nie wykonano też formalnego testu obciążenia przy ok. 10 000 rekordach w katalogu i ok. 50 RPS, który był zapisany w wymaganiach jakościowych.

Zamiast tego 25.10.2026 zrobiono pomiary Grafana k6 na Oracle VM [8]. Dla listy wydarzeń komfort to ok. 150–200 RPS; przy 50 RPS nie było błędów (p95 ok. 4,5 ms). Health API i statyczny front trzymają ok. 2000 RPS w strefie komfortu. Wąskim gardłem jest MySQL, a nie nginx. Ponieważ k6 działał na tej samej maszynie co aplikacja, wynik traktujemy jako dolną granicę — to nadal nie jest formalny scenariusz 10k rekordów / 50 RPS.

Na potrzeby MVP i obrony taki obraz wydajności wystarcza. Na później zostają m.in. płatności online, pełniejsza mobilka, formalny test 10k/50 RPS oraz dalsze utwardzenie środowiska produkcyjnego.

Bibliografia

[1] „React Documentation”. https://react.dev/. dostęp: 2026-11-07.

[2] „Spring Boot Reference Documentation”. https://docs.spring.io/spring-boot/docs/current/reference/html/. dostęp: 2026-11-07.

[3] „MySQL 8.4 Reference Manual”. https://dev.mysql.com/doc/refman/8.4/en/. dostęp: 2026-11-07.

[4] „Expo Documentation”. https://docs.expo.dev/. dostęp: 2026-11-07.

[5] „RacePortal — repozytorium projektu”. https://github.com/Gucio-009/RacePortal. Branch main; dostęp: 2026-11-07.

[6] „JSON Web Token (JWT)”. https://datatracker.ietf.org/doc/html/rfc7519. RFC 7519; dostęp: 2026-11-07.

[7] „OWASP Application Security Verification Standard”. https://owasp.org/www-project-application-security-verification-standard/. dostęp: 2026-11-07.

[8] „Wyniki load testu k6 — Oracle VM (RPS)”. dokumentacja projektu RacePortal (docs/testy/wyniki/k6-vm-rps.md). sesja pomiarowa 2026-10-25; dostęp: 2026-11-07.

[9] „Vite Documentation”. https://vite.dev/guide/. dostęp: 2026-11-07.

[10] „Docker Compose overview”. https://docs.docker.com/compose/. dostęp: 2026-11-07.

[11] „Open Source Routing Machine”. http://project-osrm.org/. dostęp: 2026-11-07.

[12] „Leaflet — an open-source JavaScript library for mobile-friendly interactive maps”. https://leafletjs.com/. dostęp: 2026-11-07.

[13] „Grafana k6 documentation”. https://grafana.com/docs/k6/latest/. dostęp: 2026-11-07.

[14] „LaTeX — A document preparation system”. https://www.latex-project.org/. dostęp: 2026-11-07.

Spis rysunków

4.1 Podział interesariuszy projektu — funkcjonalności dostępne dla ról użytkowników (materiał koncepcyjny). 33

7.1 Diagram przypadków użycia — Platforma motorsportowa (wersja koncepcyjna). 59

8.1 Diagram klas / model danych (ERD) — wersja koncepcyjna projektu. 63

8.2 Prototyp UI RacePortal w Figma Make — strona główna (ciemny motyw motorsportowy, akcent marki). 67

Spis tabel

4.1 Karta udziałowca: Gość / kibic 34
4.2 Karta udziałowca: Kierowca 34
4.3 Karta udziałowca: Organizator 35
4.4 Karta udziałowca: Administrator 35
4.5 Karta udziałowca: Właściciel toru / partner obiektu 36
4.6 Porównanie modelu grup społecznościowych z modelem RacePortal (ujęcie koncepcyjne). 36

6.1 Zakres MVP — funkcje z KP/DZW oraz stan według `docs/MVP.md` (28.10.2026). 47
6.2 Podział ról według DZW (formalny dla tego rozdziału) oraz opis w `README.md`. 48
6.3 Etapy prac — plan z Harmonogramu PRO wobec charakteru przebiegu w repozytorium. 49
6.4 Analiza ryzyka RacePortal — wybrane zagrożenia i sposoby ich ograniczenia. . 51

7.1 Zintegrowany ekosystem wydarzeń motorsportowych 53
7.2 Automatyzacja obiegu zgłoszeń i weryfikacja pojazdów 53
7.3 Przejrzysty podział ról w systemie 53
7.4 Publiczny katalog i inteligentne filtrowanie 54
7.5 Szczegóły wydarzenia i wyznaczanie trasy 54
7.6 Zarządzanie profilem i kontem 54
7.7 Wirtualny Garaż Kierowcy 55
7.8 Szybkie aplikowanie na wydarzenia 55
7.9 Skuteczny system powiadomień 55
7.10 Panel Zarządzania dla Organizatora 55
7.11 Panel Administratora i wsparcie użytkowników 56
7.12 Mobilna aplikacja wizytówkowa 56
7.13 Wydajność i responsywność platformy 56
7.14 Dostępność wieloplatformowa (RWD) 57
7.15 Niezawodność i łatwość wdrażania 57
7.16 Bezpieczeństwo danych osobowych i kont 57
7.17 Ścisła kontrola dostępu do zasobów 58

7.18 Odporność na nadużycia platformy 58
7.19 Nowoczesny design branżowy 58
7.20 Intuicyjność i czytelność procesów 59

Załączniki

Płyta CD z następującą zawartością:

- *pliki projektowe* – pliki składające się na całość projektu
  - repozytorium kodu źródłowego wraz z instrukcją zbudowania i uruchomienia projektu
  - źródło pracy inżynierskiej.
- *Otrombke Natalia_Kasprowicz Oliwier_Gutowski Michał_Parkitny Miłosz_Wronisz Wojciech_praca pisemna* – katalog zawierający plik PDF z pracą inżynierską.